

# ***Relazione Progetto Cognitive Computing and Artificial Intelligence***

*Anno Accademico 2025/2026*

## ***AutomotiveBloggerAgent***

## Sommario

|                                                                   |    |
|-------------------------------------------------------------------|----|
| Introduzione.....                                                 | 3  |
| Obiettivi e specifiche del progetto .....                         | 3  |
| Scelte progettuali e Stack tecnologico .....                      | 4  |
| Architettura del sistema.....                                     | 8  |
| LangGraph e i grafi a stati.....                                  | 8  |
| Stato condiviso .....                                             | 9  |
| Fasi dell'agente.....                                             | 9  |
| Gli strumenti dell'agente .....                                   | 11 |
| Motivazioni delle scelte architetturali .....                     | 12 |
| Knowledge Graph .....                                             | 12 |
| Introduzione al grafo di conoscenza editoriale.....               | 12 |
| Implementazione del grafo .....                                   | 13 |
| Strumenti di accesso al grafo .....                               | 14 |
| La gap-analysis .....                                             | 14 |
| Il backlog delle proposte .....                                   | 14 |
| Scelte progettuali implementative del KG e limiti incontrati..... | 15 |
| Retrieval-Augmented Generation (RAG) .....                        | 17 |
| Introduzione al RAG .....                                         | 17 |
| Funzionamento del RAG .....                                       | 17 |
| Utilizzi avanzati del RAG nel progetto .....                      | 18 |
| Implementazione nel progetto .....                                | 18 |
| Motivazioni delle scelte progettuali e limiti incontrati.....     | 19 |
| ReAct e Planning .....                                            | 20 |
| Introduzione al ReAct .....                                       | 20 |
| Introduzione al Planning .....                                    | 21 |
| Implementazione del ReAct e Planning nel progetto .....           | 21 |
| Motivazioni delle scelte implementative e limiti incontrati.....  | 23 |
| Human-in-the-Loop (HITL) .....                                    | 27 |
| Introduzione all'HITL .....                                       | 27 |
| I cinque punti di Human-in-the-Loop .....                         | 27 |
| Motivazioni delle scelte implementative e limiti incontrati.....  | 29 |

|                                                                   |    |
|-------------------------------------------------------------------|----|
| Model Context Protocol (MCP) .....                                | 29 |
| Introduzione all'MCP .....                                        | 29 |
| Implementazione dell'MCP nel progetto .....                       | 30 |
| Motivazioni delle scelte implementative e limiti incontrati ..... | 31 |
| Architettura multi-modello .....                                  | 32 |
| La sfida dei modelli locale e le scelte adottate .....            | 32 |
| Architettura Ibrida .....                                         | 33 |
| Motivazioni delle scelte progettuali e limiti incontrati .....    | 33 |
| Fine-Tuning .....                                                 | 34 |
| Introduzione al Fine-Tuning .....                                 | 34 |
| Sviluppo del training e implementazione nel progetto .....        | 35 |
| Valutazione dei risultati del Fine Tuning .....                   | 37 |
| Motivazioni delle scelte progettuali e limiti incontrati .....    | 39 |
| Evaluation e Observability .....                                  | 42 |

## Introduzione

### Obiettivi e specifiche del progetto

Il seguente progetto “AutomotiveBloggerAgent” rappresenta il prototipo di un di un’assistente editoriale agentico per quanto riguarda il settore automotive. Il sistema supporta l’autore del blog nell’intero ciclo editoriale: suggerisce argomenti a partire dalla memoria storica del blog e delle notizie fresche di giornata, pianifica sequenze di post motivandone le ragioni, conduce ricerche sul web e locali per giustificare le fonti, redige bozze attinenti alle fonti raccolte e mantiene una memoria di tutti i contenuti pubblicati e approvati in un Knowledge Graph (KG). Inoltre, ogni passaggio decisionale rilevante è soggetto ad approvazione umana, tramite l’HITL (Human-in-the-Loop).

Ogni scelta architetturale risponde ai requisiti delle specifiche richieste, in cui veniva chiesto di realizzare un sistema agentico dotato di:

- Capacità di verifica e selezione delle fonti
- Knowledge Graph attivamente usato per tenere traccia dei post approvati
- Retrieval-Augmented Generation (RAG) per fornire un grounding alle bozze redatte
- ReAct secondo lo stile del “Thoughts, Actions, Observations”

- Human-in-the-Loop (HITL) per la “revisione” di ogni contenuto generato dall’agente
- Model Context Protocol (MCP) applicato ad un tool
- Implementazione di almeno due tool, di cui uno soggetto a Fine-Tuning.

## Scelte progettuali e Stack tecnologico

Tra le scelte caratterizzanti del progetto spicca l’esecuzione locale dei modelli linguistici tramite Ollama. In particolar modo sono stati impiegati due modelli:

- Minstral 3 (3B): Rappresenta il cervello del sistema nonché la “penna”: orchestra tutta la fase di pianificazione, briefing, ReAct, sintesi delle fonti dalla ricerca web e la stesura finale del post.
- Llama 3.2 (1B) Fine Tuned tramite QLoRA: Impiegato in uno dei tool usati dall’agente, come “giudice” nel confronto di due modelli di veicoli (auto o moto).

Questa scelta ha reso il progetto particolarmente complesso da realizzare in quanto è stato sviluppato principalmente tenendo conto dei limiti presenti nei modelli di piccole dimensioni. Molte scelte architetturelle che verranno spiegate più avanti nascono dall’esigenza di rendere “affidabile” i comportamenti del modellino utilizzato. I problemi incontrati e le relative soluzioni adottate dimostrano le difficoltà nella costruzione di sistemi agentici con modelli “limitati” a livello di risorse.

Passando alle scelte tecnologiche chiave, troviamo:

- LangGraph: Framework di orchestrazione a grafo dell’agente.
- Ollama: Strumento open-source che ha permesso l’esecuzione locale dei modelli citati.
- Neo4J (Aura DB): Servizio Cloud (SaaS) per l’implementazione del Knowledge Graph (database a grafo).
- ChromaDB: Database Vettoriale open-source, utilizzato nel RAG, per il salvataggio di documenti locali sottoforma di embeddings vettoriali.
- MCP (Model Context Protocol): Standard open-source introdotto da Anthropic che permette agli LLM di connettersi a servizi esterni (come tools).
- LangSmith: Piattaforma di osservabilità completa dell’agente
- Cloudflare Workers AI: Piattaforma che permette di eseguire modelli AI direttamente sulla rete di Cloudflare.
- Kaggle: Ambiente di sviluppo che mette a disposizione risorse utilizzate nell’ambito dell’addestramento (Fine Tuning o Full Tuning) di modelli.
- Tavily: motore di ricerca progettato per l’AI, specificatamente per gli agenti.

# Struttura del progetto

## Introduzione alla struttura implementativa

Introduciamo il progetto a partire dall'organizzazione del codice, descrivendo cartella per cartella e file per file, cosa contiene e perché. Ogni cartella isola un sottosistema (il grafo dell'agente, il Knowledge Graph, il RAG, i tool, i prompt, il servizio MCP, la valutazione), così che ogni componente sia leggibile, testabile e modificabile in modo indipendente.

## Root del progetto

- **main.py**: entry point da riga di comando. Avvia il grafo e, soprattutto, gestisce le interazioni Human-in-the-loop: quando il grafo si mette in pausa su un interrupt, main.py lo smista all'handler giusto in base all'azione (clarify\_request, review\_editorial\_plan, review\_post\_draft, continue\_writing, choose\_suggestion), gestendo anche catene di pause concatenate. Ogni richiesta usa un thread\_id nuovo, così lo stato è isolato per task.
- **utils.py**: utility condivisa per il Self-RAG: definisce lo schema GradeDocuments (giudizio binario yes/no sulla rilevanza) e la whitelist GRADABLE\_TOOLS, che stabilisce quali tool vanno valutati prima dell'uso (solo la ricerca web, l'unica fonte "variabile").

## Configurazione

- **settings.py**: un unico file di Configuration che centralizza tutti i parametri: nomi dei modelli (ministral-3:3b), provider Ollama, temperature per ruolo (0.0 / 0.6 / 0.2), finestre di contesto (22528 e 22528), parametri RAG (top-k 5, soglie 1.10 e 1.17), e flag di debug. Centralizzare qui evita "numeri magici" sparsi nel codice.

## Agent

È il cuore del sistema. Separa nettamente lo stato, la struttura del grafo, la logica dei nodi, le decisioni di instradamento e le utility.

- **state.py**: definisce lo State (che estende MessagesState) e tutti gli schemi Pydantic strutturati: ClarifyWithUser, ResearchBrief, KGExtraction, PostPlan/PlanningSchema (pianificazione), ProposalAction/EditorialDecision (gate editoriale). È il "contesto" esplicito che attraversa tutto il grafo.
- **graph.py**: assembla nodi ed edge e compila il grafo con un MemorySaver come checkpoint. Espone build\_graph() (con opzione senza checkpoint per LangGraph Studio) e l'istanza graph usata da main.py.
- **nodes.py**: indica la logica di ogni nodo, scoping (clarification, brief), contesto KG, planner, gate editoriale, ricerca (research agent + tool node resiliente + ricerca forzata + riscrittura query), stesura, revisione, aggiornamento KG, gate "prossimo post". È il file più corposo e contiene anche i guardrail deterministici (limiti di ricerca, blocco chiamate ripetute, prefisso Observation sui risultati).

- **routing.py**: indica dove andare dopo ogni nodo. Contiene il tetto `MAX_RESEARCH_STEPS = 10`, il routing del ciclo ReAct (verso i tool, la stesura, o la ricerca forzata) e `grade_documents`, il nodo di Self-RAG che valuta la pertinenza delle fonti web.
- **helpers.py**: è una utility deterministiche che "aiutano" il modello 3B a non sbagliare, contiene `wants_post` (post vs suggerimenti), `extract_num_posts` (quanti post pianificare, default 3, cap 5), `is_clearly_vague` (scoping), `canonical_topic` (chiave normalizzata per il KG), `normalize_tool_args` (corregge i nomi dei parametri sbagliati dal modello), `has_collected_sources` (guardrail fonti), `strip_reasoning_preamble` (pulisce il ReAct dalla bozza).
- **llm.py**: inizializza i client Ollama, contiene `llm` (il "cervello": `planning`, `tool`, `grading`, `routing`) e `drafting_llm` (la "penna": solo stesura, temperatura più alta). Entrambi con `keep_alive=0` (scaricati dalla VRAM dopo l'uso).

## Prompts

Tutti i prompt sono isolati qui, separati dalla logica, così si possono rifinire senza toccare il codice.

- **system\_prompts.py**: il `blogger_system_prompt` (identità dell'agente, formato ReAct, le 5 fasi operative, regole negative) più i blocchi `background` e `editorial_guidelines` che definiscono il dominio e il tono del blog "AutomotiveAI".
- **agent\_prompts.py**: il `RESEARCH_KICKOFF` (il messaggio operativo che apre la fase di ricerca, con la guida tassativa alla scelta dei tool e il formato ReAct) e gli altri prompt dei nodi.
- **tool\_prompts.py**: le descrizioni dei tool (quelle che il modello legge per decidere quando usarli) e i prompt interni dei tool (es. il profilo veicolo del compare, il riassunto delle fonti, il prompt SEO, quello di generazione immagini).

## Tools

- **base.py**: il registro centrale dei tool: `get_react_tools()` (i 5 tool che il modello sceglie nel ciclo ReAct) e `get_all_tools()` (i 5 più i 6 deterministici), con i relativi lookup per nome.
- **mcp\_client\_tool.py**: il client del servizio MCP: espone `mcp_web_search`, che dialoga via HTTP con il server di ricerca.
- **kg\_tool.py**: i 4 tool del Knowledge Graph (`query_knowledge_graph`, `list_blog_topics`, `get_editorial_context`, `update_knowledge_graph`), wrapper LangChain sulla logica in `knowledge_graph/`.
- **rag\_tool.py**: `retrieve_local_documents`, wrapper sul retriever ChromaDB.
- **specs\_tool.py**: fornisce una scheda tecnica e storia di un singolo veicolo, da API-Ninjas (auto e moto) con fallback su Wikipedia in italiano; gestisce i brand a due parole e i modelli con cifre attaccate.

- **compare\_vehicles\_tool.py**: il confronto strutturato tra due veicoli, che usa il modello fine-tuned come giudice (è il tool basato sul fine-tuning richiesto dalle specifiche).
- **trend\_tool.py**: legge i feed RSS di testate automotive italiane per i suggerimenti/attualità, con gestione robusta dei feed irraggiungibili.
- **seo\_tool.py**: esegue l'analisi della densità della keyword e leggibilità con l'indice Gulpease (adatto all'italiano), usato in rifinitura.
- **image\_tool.py**: genera la copertina del post via Cloudflare FLUX (qualità alta) con fallback su Pollinations.

## Knowledge\_Graph

- **client.py**: infrastruttura di connessione a Neo4j AuraDB (driver, sessioni, lettura), con silenziamento delle notifiche informative e utility di formattazione date.
- **queries.py**: le query Cypher in lettura: cronologia di un topic, panoramica della copertura per la gap-analysis, e kg\_topic\_context (contesto editoriale per coerenza e cross-link); include resolve\_topic per il match semantico dei topic.
- **updater.py**: contiene le scritture, update\_kg\_data (crea/aggiorna Post, Topic, Source, Claim e relazioni) e la gestione del backlog di proposte (add\_proposals, remove\_proposal) come nodi :Proposal crash-safe.
- **semantic.py**: la deduplica semantica dei topic, trasforma i topic in vettori con l'embedding all-MiniLM-L6-v2 (lo stesso del RAG) e li confronta per similarità coseno (soglia 0,75), così argomenti scritti in modo leggermente diverso non creano duplicati nel grafo.
- **seed.py**: popola il KG con alcuni post-esempio realistici, per dare all'agente una "storia editoriale" di partenza su cui fare la gap-analysis.

## RAG

- **vectorstore.py**: contiene una cartella di persistenza ./chroma\_db, collection blog\_documents, modello di embedding all-MiniLM-L6-v2, e gli oggetti embeddings/vectorstore riusati altrove.
- **ingest.py**: indicizza i .txt della cartella blog\_sources/: chunking ricorsivo (500 caratteri, overlap 50) ed embedding in ChromaDB.
- **retriever.py**: recupero con punteggio di distanza, soglia primaria (1.10) e rete di sicurezza (fallback fino a 1.17), tracciato su LangSmith.

## MCP\_Service

**search\_server.py**: il server MCP autonomo (FastMCP, porta 8765, transport streamable-http) che espone search\_and\_summarize: incapsula la ricerca Tavily, il filtro/deduplica e la sintesi per-fonte con Ministral, restituendo testo già pulito. Gira come processo separato rispetto all'agente.

## Evaluation

- **crea\_dataset.py** e **crea\_dataset\_v2.py**: creano su LangSmith i due dataset di valutazione (5 e 25 prompt), il secondo pensato per coprire tutti i rami dell'agente.
- **evaluate\_blog.py**: esegue l'agente su un dataset e applica i quattro evaluator (qualità, grounding, failure cases, uso dei tool), con Gemini 3.1 Flash Lite come giudice (fallback locale), e produce il report di osservabilità.

## Architettura del sistema

### LangGraph e i grafi a stati

L'intera architettura del progetto è stata costruita con LangGraph, un framework per realizzare applicazioni agentiche modellandole come grafi a stati. L'idea di fondo è rappresentare il comportamento dell'agente come un grafo composto da nodi e archi. Un nodo è una funzione che svolge un pezzo di lavoro (per esempio pianificare, invocare uno strumento, scrivere la bozza); un arco è una transizione che collega un nodo al successivo. Attraverso il grafo scorre uno **stato condiviso**, cioè una struttura dati che contiene tutte le informazioni della conversazione in corso, che ogni nodo può leggere e aggiornare.

La caratteristica che rende un grafo più potente di una semplice catena lineare di passaggi è la possibilità di avere archi condizionali: la transizione successiva può dipendere dal contenuto dello stato. Questo abilita tre comportamenti fondamentali per un agente:

- **Cicli**: ad esempio una ripetizione della ricerca finché le fonti raccolte non siano sufficienti.
- **Diramazioni**: cioè percorsi diversi a seconda della situazione o richiesta dell'utente (es. la scrittura di un post è un ramo diverso dai suggerimenti)
- **Pause**: ovvero sia dei punti nel grafo in cui l'esecuzione viene fermata attraverso gli interrupt per aspettare l'intervento umano (con l'HITL).

La modellazione di un'agente editoriale come un grafo è una scelta essenziale se si pensa al lavoro editoriale, l'agente infatti può aver bisogno di effettuare ricerche multiple, strade diverse, tool diversi a seconda della situazione; tutto ciò non è possibile attraverso una sequenza statica di operazioni prefissate.

Concentrandoci sulle pause, centrale per il funzionamento è lo Human-in-the-Loop, in cui l'esecuzione viene appunto sospesa momentaneamente (con degli interrupt) e ripresa esattamente nel punto in cui è stata fermata a seguito della risposta dell'utente, senza perdere il contesto grazie a dei checkpoint. Lo stesso meccanismo rende il sistema resiliente, perché lo stato persiste tra un passo e l'altro invece di vivere solo in variabili volatili.



## Stato condiviso

Il passaggio dei dati da un nodo all'altro è possibile grazie al concetto di stato. Permette ai nodi di comunicare senza variabili globali. Lo stato raccoglie, tra le altre cose: la richiesta dell'utente, la traccia del ragionamento ReAct (`reasoning_trace`), gli output dei tool e le note grezze, la sintesi di contesto dal Knowledge Graph, le informazioni di pianificazione (proposte, post selezionati, post corrente, numero di post richiesti, temi scartati), le fonti raccolte (locali e web), la bozza, il feedback umano, e una serie di contatori e flag che governano i guardrail (numero di ricerche web effettuate, firme delle chiamate ai tool già eseguite, flag di verifica fonti, contatore di revisioni).

Ogni nodo riceve lo stato, ne legge le parti che gli servono e restituisce un aggiornamento; LangGraph fonde l'aggiornamento nello stato secondo regole di *reduce* (per esempio, i messaggi si accumulano in coda). Il passo successivo legge sempre dallo stato aggiornato, mai da memoria esterna.

## Fasi dell'agente

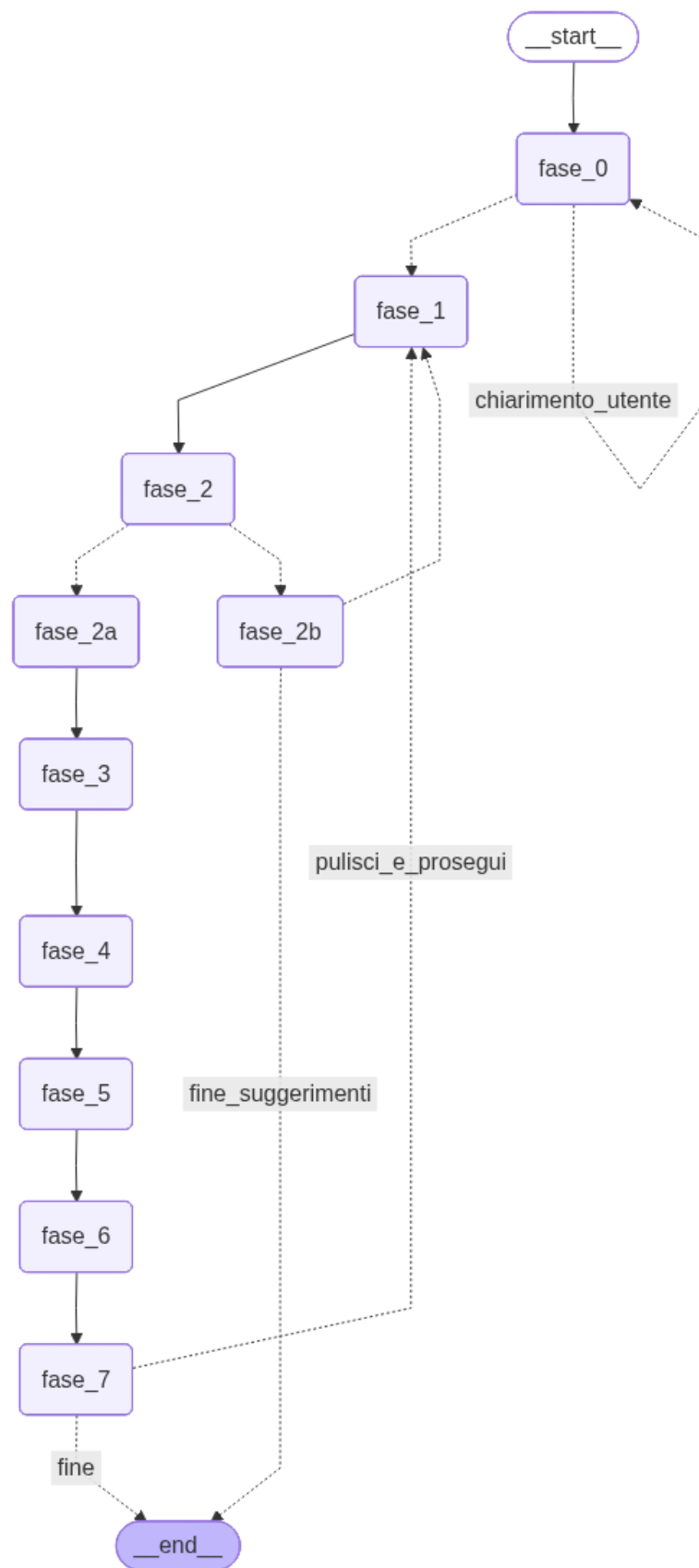
Il grafo dell'AutomotiveBloggerAgent si articola nelle fasi seguenti. Tra parentesi sono indicati i punti di Human-in-the-Loop (HITL), cioè le pause in cui il sistema attende una decisione dell'utente.

- **Fase 0: Scoping / chiarimento (HITL).** All'arrivo della richiesta, l'agente valuta se è sufficientemente chiara. Un guardrail di sicurezza deterministica intercetta le richieste palesemente generiche ("scrivimi qualcosa", "parlami di auto") e fa scattare una richiesta di chiarimento all'utente (al massimo due volte, per non avvitarsi). Se la richiesta è già chiara, si prosegue senza interrompere.
- **Fase 1: Brief e contesto dal Knowledge Graph.** La richiesta viene trasformata in un brief editoriale strutturato (tema, taglio, note); subito dopo l'agente recupera dalla memoria editoriale la panoramica di copertura del blog: cosa è già stato pubblicato, quali argomenti sono poco coperti (*gap-analysis*), quali proposte sono rimaste in sospeso.
- **Fase 2: Pianificazione.** Il planner propone un piccolo calendario editoriale di N post (il numero richiesto dall'utente, con un valore predefinito altrimenti pari a 3), ciascuno con categoria, motivazione e giustificazione dell'ordine, sfruttando la *gap-analysis* per privilegiare gli argomenti meno trattati. A valle, un arco condizionale instrada verso uno di due rami:
  - se la richiesta è di scrittura, si va al gate del piano editoriale;
  - se la richiesta è di soli suggerimenti (es. "di cosa parliamo oggi?"), si va al ramo dei suggerimenti.
- **Fase 2a: Revisione del piano editoriale (HITL).** Le proposte vengono presentate all'utente, che può approvarle tutte, selezionarne un sottoinsieme, modificarne

alcune, scartarne, oppure chiederne di nuove. Le proposte confermate vengono salvate nel Knowledge Graph come proposte in sospeso prima di iniziare a scrivere, così nessun lavoro di pianificazione va perso in caso di interruzione.

- **Fase 2b: Suggerimenti (HITL, ramo alternativo).** Nel ramo "di cosa parliamo oggi?" l'agente presenta, in ordine di priorità, le proposte in sospeso recuperabili dal grafo, il nuovo calendario editoriale e le notizie fresche dai feed RSS; poi chiede se l'utente vuole scriverne uno. Se l'utente sceglie un tema, il flusso rientra nella scrittura (ripartendo dal brief); altrimenti si chiude lasciando i suggerimenti come risposta.
- **Fase 3: Ricerca (ciclo ReAct).** L'agente sceglie e usa gli strumenti più adatti per raccogliere informazioni, alternando ragionamento e azione. Un meccanismo di auto-valutazione delle fonti (*Self-RAG*) giudica se i risultati sono pertinenti e, in caso contrario, riformula la ricerca; alcuni guardrail deterministici evitano cicli infiniti, chiamate ripetute e la stesura senza alcuna fonte.
- **Fase 4: Stesura.** Il modello redige la bozza dell'articolo in Markdown, fondandola sulle fonti raccolte e sul contesto di coerenza estratto dal grafo, con regole tassative di citazione e grounding.
- **Fase 5: Revisione della bozza (HITL).** Il grafo si ferma e presenta la bozza all'utente, che può approvare, modificare (con un feedback che innesca la rigenerazione) o scartare.
- **Fase 6: Pubblicazione.** Solo dopo l'approvazione, l'articolo viene pubblicato: si aggiorna il Knowledge Graph con post, fonti, claim e relazioni; si genera l'immagine di copertina; si calcolano le metriche SEO. Questi strumenti non sono scelti dal modello, ma invocati in modo deterministico in questa fase.
- **Fase 7: Prossimo post (HITL, ciclo multi-post).** Se erano stati pianificati più post, un gate di avanzamento permette di continuare con il successivo, sceglierne uno in particolare o fermarsi. Tra un post e l'altro lo stato viene ripulito (messaggi, contatori, fonti) mantenendo però la coda dei post selezionati.

Di seguito viene mostrata un'immagine che riassume l'intero grafo.



## Gli strumenti dell'agente

Durante la fase di ricerca l'agente sceglie, secondo il paradigma ReAct, tra un insieme di strumenti: la ricerca web tramite MCP (`mcp_web_search`), il recupero dai documenti locali

(retrieve\_local\_documents, il RAG), l'interrogazione del Knowledge Graph, l'estrazione della scheda tecnica di un veicolo (fetch\_vehicle\_specs), il confronto strutturato tra due veicoli (compare\_vehicles, basato sul modello fine-tuned) e l'analisi delle tendenze dai feed RSS (fetch\_automotive\_trends).

Altri strumenti non sono lasciati alla scelta del modello, ma invocati deterministicamente al momento giusto: la scrittura nel Knowledge Graph (solo dopo l'approvazione), la generazione della copertina e l'analisi SEO (in fase di pubblicazione). Questa distinzione ha permesso di ridurre il numero di strumenti, tra cui il modello deve scegliere durante il ReAct rende la selezione più affidabile per un modello di piccole dimensioni, riservando l'automazione deterministica ai passi in cui non serve alcuna decisione.

## Motivazioni delle scelte architetturali

Dare a ogni fase un nodo, un prompt e una responsabilità dedicati produce tre vantaggi concreti: permette di inserire guardrail mirati nei punti giusti senza appesantire il resto; rende il tracing leggibile (su LangSmith ogni nodo è un passo distinto, con i suoi token e la sua latenza); e consente interventi precisi quando un singolo passaggio si comporta come non dovrebbe, senza toccare l'intero sistema.

La seconda scelta di fondo è affidare le decisioni dell'utente a parser deterministici invece che al modello: le risposte ai gate (selezione delle proposte, scelta del prossimo post, scelta dai suggerimenti) sono interpretate da regole esplicite, non da un LLM. La motivazione è empirica ed è documentata nei capitoli successivi: un modello di piccole dimensioni sbagliava sistematicamente l'interpretazione di comandi come "scarta il 3" o "modifica il 2", con effetti distruttivi sul piano editoriale. Il parsing deterministico è affidabile, testabile e a costo zero.

## Knowledge Graph

### Introduzione al grafo di conoscenza editoriale

Un Knowledge Graph è un modo di rappresentare l'informazione come una rete di entità collegate da relazioni. Gli elementi fondamentali sono due: i **nodi**, che rappresentano entità o concetti (una persona, un prodotto, un articolo) e possono avere delle proprietà, e gli **archi** (o relazioni), che collegano i nodi e sono tipizzati e direzionati, cioè hanno un significato preciso e un verso (per esempio "l'articolo A tratta la frenata rigenerativa", "il topic X è correlato a topic Y").

Per capire cosa rende speciale un grafo, conviene confrontarlo con le altre due strutture dati che si usano comunemente per memorizzare informazioni:

- In un **database relazionale** (SQL) i dati stanno in tabelle e le relazioni si ottengono con le join tra tabelle, basati su chiavi. Lo schema è rigido e le relazioni sono, in un certo senso, implicite: esistono ma vanno ricostruite a ogni interrogazione.

- In un **database vettoriale** i dati sono salvati come embedding (vettori numerici) e si recuperano per similarità semantica. È perfetto per rispondere a "cosa assomiglia a questo?", ma non conosce alcuna struttura esplicita: non sa dirti come due cose sono collegate, solo quanto sono simili.
- In un **Knowledge Graph**, invece, le relazioni sono **First Class Citizens**: sono esse stesse dati, esplicite e navigabili. Questo permette di interrogare la struttura ("quali articoli sono collegati alle auto elettriche?", "quali argomenti non sono ancora stati trattati?") e di percorrere il grafo seguendo le connessioni.

Tecnologicamente, il database a grafo più diffuso è Neo4j, che usa un linguaggio di interrogazione chiamato Cypher: invece di scrivere JOIN tra tabelle, si descrivono dei pattern di nodi e relazioni (per esempio MATCH (p:Post)-[:COVERS\_TOPIC]->(t:Topic)). Neo4j adotta il modello detto property graph, in cui sia i nodi sia le relazioni possono portare proprietà.

I Knowledge Graph sono importanti per i sistemi di intelligenza artificiale perché forniscono una memoria strutturata e interrogabile e permettono di ragionare sulle relazioni tra le informazioni. Nei sistemi RAG più avanzati, l'approccio chiamato GraphRAG combina proprio il recupero di informazioni con la struttura di un grafo, il quale è stato implementato e verrà spiegato nel capitolo successivo.

## Implementazione del grafo

Nell'AutomotiveBloggerAgent il Knowledge Graph rappresenta la memoria editoriale del blog: al suo interno sono registrati gli articoli che sono stati pubblicati e proposti. Il grafo è ospitato su Neo4j Aura, la versione gestita cloud di Neo4j.

Lo schema è centrato sui post pubblicati ed è composto da cinque tipi di nodo e quattro tipi di relazione:

- **:Post** — l'articolo pubblicato. Proprietà: title, category (la categoria editoriale: *how\_to*, *review*, *news*, *events*), created\_at (timestamp), content, e inoltre seo\_score e cover\_image, salvati in fase di pubblicazione.
- **:Topic** — un argomento. Proprietà: name.
- **:Source** — una fonte usata per scrivere il post. Proprietà: url.
- **:Claim** — un'affermazione chiave estratta dal post. Proprietà: text.
- **:Proposal** — una proposta editoriale approvata ma non ancora scritta. Proprietà: topic\_key (chiave canonica), title, category, justification, created\_at.

Le relazioni collegano i post agli altri nodi:

- (:Post)-[:COVERS\_TOPIC]->(:Topic) — il post tratta quel topic;
- (:Topic)-[:RELATED\_TO]->(:Topic) — due argomenti sono correlati;
- (:Post)-[:BASED\_ON]->(:Source) — il post si basa su quella fonte;

- `(:Post)-[:ASSERTS]->(:Claim)` — il post afferma quel concetto chiave.

## Strumenti di accesso al grafo

L'agente interagisce con il Knowledge Graph attraverso quattro strumenti, tre di lettura e uno di scrittura:

- **query\_knowledge\_graph** restituisce la cronologia di un topic specifico (è già stato trattato? quando? con quali correlati?) e serve a evitare doppioni.
- **list\_blog\_topics** fornisce la panoramica completa della copertura editoriale, con i topic ordinati dal più trascurato. È lo strumento che alimenta la gap-analysis.
- **get\_editorial\_context** recupera il contesto completo di un topic (post esistenti, claim già affermati, fonti già usate, argomenti correlati) e viene usato durante la stesura per garantire coerenza.
- **update\_knowledge\_graph** scrive nel grafo un nuovo nodo-post, ma soltanto *dopo* che l'utente ha approvato la bozza finale.

Oltre a questi strumenti, alcune interrogazioni del grafo sono usate direttamente dai nodi del grafo, non come tool scelti dal modello: per esempio il recupero dei titoli dei post recenti che alimenta il pianificatore, l'elenco delle proposte in sospeso che alimenta i suggerimenti, e l'espansione della query di retrieval con i topic correlati. La separazione è coerente con la filosofia generale del sistema: ciò che richiede una decisione è un tool a disposizione del modello, ciò che è deterministico viene invocato direttamente.

## La gap-analysis

Il meccanismo più importante è l'analisi dei vuoti di copertura: nella fase di pianificazione l'agente interroga il grafo per capire cosa è già stato coperto e da quanto tempo, e propone nuovi post che colmano i vuoti, argomenti mai trattati o trattati molto tempo prima. È qui che la "memoria" del grafo diventa utile per guidare la strategia editoriale invece di reagire alla singola richiesta in modo isolato.

## Il backlog delle proposte

Quando l'utente approva un piano editoriale al gate, le proposte selezionate vengono salvate subito come nodi `:Proposal`, prima che inizi la scrittura. Questo rende il piano sicuro da errori: se l'agente si interrompe a metà di una sequenza di post, le proposte non ancora scritte restano nel grafo e vengono ripresentate tra i suggerimenti alla sessione successiva. Quando un post viene pubblicato, la proposta corrispondente viene rimossa e promossa a "Post".

A corredo, lo script `seed.py` popola il grafo con alcuni post di esempio (così, durante una dimostrazione, la gap-analysis ha una storia su cui ragionare invece di partire da un grafo vuoto).

## Scelte progettuali implementative del KG e limiti incontrati

La decisione progettuale più importante è stata definire il Knowledge Graph come memoria di ciò che è stato pubblicato, e non come una lista di cose da fare. Era emersa la proposta alternativa di popolare il grafo con gli argomenti ancora da trattare. Il problema è che la gap-analysis del pianificatore ragiona proprio sul contenuto del grafo: se lo si riempisse di argomenti futuri, il pianificatore li interpreterebbe come già pubblicati e quindi smetterebbe di proporli, ottenendo l'esatto contrario del comportamento desiderato. La semantica conta: "memoria del pubblicato" e "piano del futuro" sono due concetti diversi, e mescolarli rompe la logica del sistema.

Questa filosofia si riflette fin nel dettaglio dello schema. Il backlog delle proposte è stato modellato in modo da non inquinare la gap-analysis: il nodo `:Proposal` tiene il proprio argomento come semplice proprietà (`topic_key`) e non crea nodi `:Topic` né relazioni `COVERS_TOPIC`. In questo modo la gap-analysis — che misura la copertura contando i `:Post` collegati ai `:Topic` — "vede" soltanto ciò che è stato realmente pubblicato, e una proposta ancora da scrivere non viene mai scambiata per un argomento già coperto. È la traduzione, a livello di codice, della distinzione tra memoria del pubblicato e piano del futuro.

Passando ai limiti invece, il primo fra tutti riguarda l'operazione di inserimento di post all'interno del grafo. Per registrare un post nel grafo, il modello deve estrarre i parametri corretti (topic, categoria, fonti, claim, correlati); un modello locale di piccole dimensioni a volte sbaglia o impoverisce questa estrazione. Il problema è mitigato (normalizzazione dei parametri e resilienza del nodo che scrive) ma non eliminato del tutto.

Secondariamente la qualità della gap-analysis dipende dalla coerenza dei topic. Se gli argomenti vengono nominati o categorizzati in modo incoerente (per esempio "auto elettriche" e "veicoli elettrici" come due topic distinti), l'analisi dei vuoti perde precisione. La normalizzazione dei nomi dei topic mitiga il problema ma non lo annulla.

Di seguito vengono mostrati due screen presi direttamente da Neo4J in cui è presente un estratto editoriale del KG attuale, con l'esempio di un post pubblicato ed uno proposto.

| Node details |                                                                                                                                                                                                |  |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| Post         |                                                                                                                                                                                                |  |
| Key          | Value                                                                                                                                                                                          |  |
| <id>         | 4:b2e13306-84ad-4bea-98f2-9826974b4e99:130                                                                                                                                                     |  |
| category     | "review"                                                                                                                                                                                       |  |
| content      | "# **[Review] Confronto Diretto: Hyundai i30N Performance vs. Hyundai i20N Performance – Prestazioni, Design e Comfort in un Duello Sportivo**<br><br>---<br><br>#... <a href="#">Show all</a> |  |
| cover_image  | "generated_covers\copertina_post_1781368080.png"                                                                                                                                               |  |
| created_at   | 2026-06-13T16:28:13.044000000Z                                                                                                                                                                 |  |
| seo_score    | 44.8                                                                                                                                                                                           |  |
| title        | "[review] confronto diretto: hvundai i30n performance vs. "                                                                                                                                    |  |

| Node details  |                                                                                                                                                                                    |  |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| Proposal      |                                                                                                                                                                                    |  |
| Key           | Value                                                                                                                                                                              |  |
| <id>          | 4:b2e13306-84ad-4bea-98f2-9826974b4e99:142                                                                                                                                         |  |
| category      | "review"                                                                                                                                                                           |  |
| created_at    | 2026-06-13T16:32:21.824000000Z                                                                                                                                                     |  |
| justification | "**Primo post obbligatorio**: Il brief richiede una recensione tecnica della **Fiat Panda 1.3 Multijet del 2008** (motore 69 CV). Questo tema è stato ... <a href="#">Show all</a> |  |
| title         | "[Review] Fiat Panda 1.3 Multijet (2008): affidabilità, consumi e prestazioni in un contesto tecnologico obsoleto ma iconico"                                                      |  |
| topic_key     | "review fiat panda multijet"                                                                                                                                                       |  |



# Retrieval-Augmented Generation (RAG)

## Introduzione al RAG

Il **RAG** (Retrieval-Augmented Generation) è una tecnica che combina un modello linguistico con il recupero di documenti esterni. Per capire perché serve, bisogna partire da un limite intrinseco dei modelli linguistici: la loro conoscenza è parametrica, cioè "congelata" nei pesi del modello al momento dell'addestramento. Da questo derivano tre problemi:

- **Allucinazioni:** quando non conosce una risposta, un modello tende comunque a produrne una plausibile ma potenzialmente falsa, perché genera il testo più probabile, non necessariamente quello vero.
- **Ignoranza dei dati privati o locali:** un modello non conosce documenti che non erano nel suo addestramento, come gli appunti tecnici personali del blogger.
- **Informazioni non aggiornate:** non sa nulla di eventi successivi alla data del suo addestramento. Di conseguenza potrebbe tranquillamente "allucinare" quando gli viene chiesto qualcosa di "nuovo".

Il RAG risolve questi problemi recuperando, al momento della domanda, i documenti più pertinenti da una base di conoscenza esterna e fornendoli al modello come contesto. In questo modo il modello genera una risposta ancorata a fonti reali invece di affidarsi solo alla memoria. Si dice che la risposta è **grounded**, cioè fondata su documenti verificabili.

## Funzionamento del RAG

Il funzionamento del RAG si articola in due momenti. Il primo è l'indicizzazione: i documenti vengono spezzati in frammenti (chunk), ogni frammento viene trasformato in un embedding, cioè un vettore numerico che ne cattura il significato semantico, e gli embedding vengono salvati in un database vettoriale.

Gli embedding sono il cuore del meccanismo: sono rappresentazioni numeriche in cui la vicinanza geometrica corrisponde alla somiglianza di significato. Due frasi che parlano della stessa cosa, anche con parole diverse, producono vettori vicini nello spazio. È ciò che permette la ricerca semantica, diversa dalla semplice ricerca per parole chiave.

Il chunking (la suddivisione in frammenti) serve per due ragioni: i documenti interi sono spesso troppo lunghi per la finestra di contesto del modello, e il recupero è più preciso su pezzi piccoli e mirati. Qui c'è un compromesso: frammenti troppo piccoli perdono il contesto, frammenti troppo grandi diventano imprecisi nel recupero. Una overlap tra frammenti consecutivi aiuta a non spezzare bruscamente i concetti a cavallo tra due chunk.

Il secondo momento è il recupero e generazione: la domanda dell'utente viene trasformata anch'essa in un embedding, il database vettoriale restituisce i frammenti più vicini (per distanza o similarità), questi vengono inseriti nel prompt e il modello genera la risposta basandosi su di essi.

## Utilizzi avanzati del RAG nel progetto

Il RAG di base ha un punto debole: recupera sempre dei frammenti e li usa, anche quando non sono davvero pertinenti. Il Self-RAG introduce un passo di auto-riflessione: il sistema valuta la rilevanza dei documenti recuperati prima di usarli e, se non sono pertinenti, può decidere di recuperare di nuovo o di riformulare la domanda. È un controllo di qualità che protegge dall'uso di contesto fuorviante.

Un secondo perfezionamento è il RAG ibrido. Il recupero vettoriale è ottimo per la somiglianza semantica su contenuti non strutturati (testi, appunti), ma non conosce relazioni esplicite. Un Knowledge Graph, al contrario, rappresenta conoscenza strutturata e relazionale. Combinare i due attraverso l'approccio reso noto con il nome di GraphRAG permette a un sistema di sfruttare insieme il contenuto semantico dei documenti e le relazioni esplicite di un grafo. Questo tipo di combinazione viene a volte chiamato K-RAG (RAG potenziato dalla “Knowledge”).

## Implementazione nel progetto

L'AutomotiveBloggerAgent usa ChromaDB come database vettoriale locale persistente. I documenti tecnici del blogger, cioè una decina di file di testo nella cartella `blog_sources/`, su argomenti come la manutenzione dei freni, il confronto tra motori termici ed elettrici, il glossario dei motori endotermici, le reti di comunicazione di bordo, i sistemi di post-trattamento, oltre a frenata rigenerativa, batterie allo stato solido, sistemi ADAS e cambi a doppia frizione; vengono indicizzati una volta sola dallo script `ingest.py`.

I parametri concreti dell'indicizzazione sono:

- **chunking:** RecursiveCharacterTextSplitter con `chunk_size = 500` caratteri e `chunk_overlap = 50`, usando come separatori, in ordine di preferenza, il doppio a-capo, il singolo a-capo, il punto e lo spazio (così la suddivisione cerca di rispettare i confini naturali del testo prima di tagliare a metà una frase);
- **embedding:** il modello `all-MiniLM-L6-v2` (un sentence-transformer leggero, eseguito in locale via HuggingFace), riusato identico sia in indicizzazione sia in recupero.

Il recupero a tempo di esecuzione (`retrieve_local`, esposto all'agente come `retrieve_local_documents`) richiede i `rag_top_k = 5` chunk più vicini alla query e applica una logica a due soglie:

- una soglia di pertinenza primaria, `rag_distance_threshold = 1.10`: vengono tenuti solo i chunk con distanza pari o inferiore (distanza più bassa = più simile);
- una soglia di fallback, `rag_fallback_max = 1.17`: se la soglia primaria non lascia passare nulla, si recupera il singolo chunk migliore, ma solo se la sua distanza è comunque entro 1.17; altrimenti non si restituisce alcun documento.

La filosofia di questa seconda soglia è "meglio non prendere nulla che prendere qualcosa di sbagliato": il fallback è un guardrail di sicurezza. I documenti locali così selezionati vengono recuperati a inizio ricerca e iniettati nel contesto di avvio del ciclo ReAct (il modello li trova già pronti, con il nome del file e la distanza, come base di conoscenza tecnica primaria).

A questo si affianca il Self-RAG: durante la ricerca, un nodo di grading (grade\_documents) giudica con un modello se le fonti raccolte sono davvero pertinenti al soggetto del post (i messaggi "Fonti rilevanti" / "Fonti non rilevanti: riformulo"); se non lo sono, un nodo di riformulazione della domanda riscrive la query e si riprova. Nella pratica questo controllo interviene soprattutto sulle fonti web raccolte nel ciclo, dove la variabilità è maggiore.

L'aspetto distintivo è la natura ibrida del retrieval. Accanto al RAG vettoriale, il sistema interroga anche il Knowledge Graph su Neo4j, e lo fa in due modi: da un lato il grafo fornisce in stesura il contesto di coerenza (claim e fonti già usati su quel topic); dall'altro la query di recupero locale viene espansa con i topic correlati presi dal grafo. Se il blog ha già trattato argomenti collegati, i loro nomi vengono aggiunti alla query prima della ricerca vettoriale, migliorando il recupero dei documenti pertinenti, implementando la combinazione "conoscenza strutturata (grafo) + contenuto non strutturato (vettoriale)"

## Motivazioni delle scelte progettuali e limiti incontrati

Il RAG è stato adottato per ancorare gli articoli a fonti reali e per sfruttare gli appunti tecnici privati del blogger, evitando che il modello scriva "a memoria" inventando dettagli.

La scelta del retrieval ibrido (vettoriale + grafo) nasce dal fatto che le due componenti rispondono a bisogni diversi: il vettoriale per la somiglianza semantica sui testi tecnici, il grafo per le relazioni strutturate della memoria editoriale.

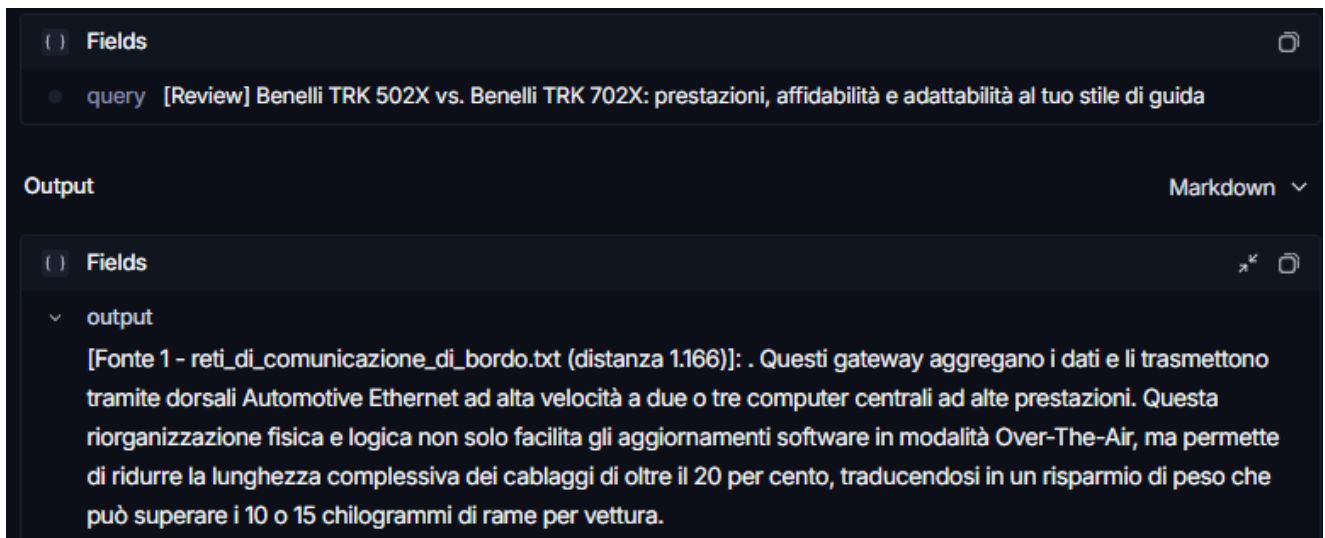
La logica a due soglie è stata introdotta perché una sola soglia non bastava: una soglia unica troppo stretta rischiava di non restituire nulla anche quando un documento utile c'era, mentre una troppo larga faceva entrare di tutto. La soglia primaria (1.10) tiene i chunk ragionevolmente vicini; il fallback (1.17) recupera al più un singolo chunk quando la primaria azzera tutto, ma solo se resta abbastanza vicino. Il Self-RAG, infine, aggiunge un filtro sul significato e non solo sulla distanza geometrica, utile soprattutto sulle fonti web.

Una soluzione alternativa all'uso della doppia soglia era l'utilizzo di un modello embedding migliore. Sono state esplorate alcune alternative come Qwen3-Embedding da 0.6B e multilingual-e5-small. La scelta di non adottare queste alternative ricade in una motivazione puramente basata su costi/prestazioni. Infatti, l'adottare un modello di embedding più grande come Qwen, avrebbe sicuramente migliorato la qualità semantica, ma al costo di un maggiore carico sul sistema e lentezza, tenendo inoltre in considerazione l'esecuzione locale dei modelli, le risorse di calcolo non sarebbero state abbastanza per eseguire un embedding più potente.

Come alternative al chunking avremmo potuto utilizzare tecniche diverse esplorate a lezione, come la suddivisione per paragrafo, integrazione del titolo o un chunking dinamico tramite

LLM. Tuttavia, non esiste una tecnica migliore in assoluto di chunking, ognuna di questa va adottata a seguito di una serie di test basati sulle query e i tipi di testi adottati.

Di seguito viene riportato uno screen preso da una run su LangSmith, in cui viene mostrato l'esecuzione del RAG. Come si può notare dalla query e dall'output ottenuto, la prima soglia ha scartato tutti i chunk di tutti i documenti poiché non erano attendibili. Mentre la seconda soglia ha catturato un chunk di un documento legato alle reti di comunicazione a bordo auto, il quale risulta completamente sconnesso dall'argomento del post, che richiama un confronto tra due modelli specifici di moto.



## ReAct e Planning

### Introduzione al ReAct

ReAct è un paradigma per costruire agenti il cui nome unisce due parole: Reasoning (ragionamento) e Acting (azione). L'idea è che il modello, invece di rispondere in un colpo solo, alterni in un **ciclo** momenti di ragionamento e momenti di azione, osservando i risultati a ogni passo. Il ciclo si compone di tre elementi che si ripetono:

- Thought (pensiero): il modello ragiona su cosa fare, su quale informazione gli manca e quale strumento potrebbe servirgli.
- Action (azione): il modello sceglie e invoca uno strumento (un tool), per esempio una ricerca web o l'interrogazione di un database.
- Observation (osservazione): il modello riceve il risultato dello strumento e lo usa per decidere il passo successivo.

Il ciclo prosegue finché il modello ha raccolto informazioni sufficienti per produrre la risposta finale. Il valore di questo approccio è che consente al modello di scomporre un compito complesso, di usare strumenti esterni per ottenere informazioni che non possiede e di adattarsi in base a ciò che osserva, invece di rispondere "a memoria" in un'unica passata. In

sostanza, ReAct combina il ragionamento passo-passo (chain-of-thought) con l'uso di strumenti.

Per rendere concreto il ciclo, ecco come potrebbe svolgersi su una richiesta di confronto tra due moto:

**Thought:** "L'utente vuole confrontare due moto. Mi serve un confronto strutturato: userò lo strumento dedicato."

**Action:** `compare_vehicles(moto_1, moto_2)`

**Observation:** [il tool restituisce il confronto in quattro categorie con verdetto]

**Thought:** "Ho il confronto, ma vorrei arricchirlo con dati di attualità sui prezzi. Cerco sul web."

**Action:** `mcp_web_search(...)`

**Observation:** [fonti web sui listini]

**Thought:** "Ho abbastanza materiale per scrivere un articolo completo. Procedo alla stesura."

Questa traccia di ragionamento è resa esplicita e tracciabile nel sistema (il `reasoning_trace`), così è possibile ricostruire perché l'agente ha scelto quei tool.

## Introduzione al Planning

Il planning è la capacità di un agente di stabilire una sequenza di passi o di sotto-obiettivi per raggiungere uno scopo. La pianificazione può avvenire tutta all'inizio ("prima pianifico, poi eseguo") oppure intrecciarsi con l'azione. In un contesto editoriale, pianificare significa decidere quali contenuti produrre e con quali priorità, ragionando in modo strategico invece di reagire alla singola richiesta in modo isolato.

## Implementazione del ReAct e Planning nel progetto

Nel progetto, planning e ReAct corrispondono a due fasi distinte.

La fase di pianificazione (Fase 2) è gestita da un nodo planner che produce un piccolo calendario editoriale, usando la gap-analysis sul Knowledge Graph per individuare gli argomenti meno coperti. Tre meccanismi deterministici ne governano il comportamento:

- Il numero di post richiesto viene estratto dalla richiesta dell'utente con una funzione dedicata (`extract_num_posts`), che riconosce cifre e numeri scritti in lettere ma ignora gli articoli ("un", "uno", "una"), così "scrivi un post" non viene interpretato come "scrivi 1 post" in senso quantitativo. Il valore predefinito è 3 e il massimo è limitato a 5.
- Le proposte vengono generate al numero richiesto (`planned[:max_posts]`): si è osservato infatti che il modello, pur ricevendo l'istruzione di generarne esattamente N, a volte ne produceva comunque 3 (per esempio rispondeva con 3 proposte a "scrivi 1 post"). Il numero dei post da generare è stato tagliato a valle, questo garantisce il conteggio richiesto senza dipendere dall'obbedienza del modello.

- Una regola di priorità garantisce che, se l'utente indica un tema preciso, il primo post sia esattamente quello, lasciando alla gap-analysis il compito di proporre i post successivi.

Le proposte sono poi sottoposte all'utente al gate editoriale (descritto nel Capitolo 6), e quelle confermate vengono salvate come backlog nel grafo prima dell'esecuzione.

La fase di ricerca (Fase 3) è il vero e proprio ciclo ReAct: il nodo research agent ragiona (Thought) e sceglie uno strumento (Action), uno nodo dei tool lo esegue e ne registra l'esito (Observation), e così via. Il formato del ragionamento è imposto in modo esplicito dal prompt di avvio (RESEARCH\_KICKOFF), che chiede al modello di scrivere, prima di ogni chiamata, una riga Thought: (perché quel tool serve adesso) e una riga Action: (quale tool con quali argomenti). Le Observation non sono lasciate alla narrazione del modello: è il nodo dei tool a scriverle nel reasoning\_trace in forma sintetica ("Observation (*nome tool*): *esito*"), così la traccia contiene l'intero ciclo Thought → Action → Observation in modo affidabile.

Durante la ricerca il modello sceglie tra cinque strumenti situazionali (quelli messi a disposizione del ragionamento ReAct): mcp\_web\_search (ricerca web tramite MCP, per attualità e validazione), compare\_vehicles (confronto strutturato tra due veicoli, basato sul modello fine-tuned), fetch\_vehicle\_specs (scheda tecnica e storia di un singolo modello), fetch\_automotive\_trends (tendenze e novità dai feed RSS) e query\_knowledge\_graph (cronologia del blog su un argomento). I documenti locali (RAG) non sono fra questi: vengono recuperati a inizio fase e iniettati direttamente nel contesto, perché su quasi ogni articolo tecnico sono utili e non c'è motivo di lasciarne la scelta al modello.

Su questo ciclo sono innestati alcuni guardrail, cioè protezioni di sicurezza deterministiche che intervengono in punti precisi senza sostituirsi alla logica del modello:

- un limite di due ricerche web per post (MAX\_WEB\_SEARCHES = 2): ogni ricerca costa diversi secondi, dai 40 ai 60 e oltre, e due sono un buon compromesso tra completezza e tempi; al raggiungimento del limite, un'eventuale ulteriore richiesta di ricerca riceve un messaggio che invita a scrivere con le fonti già raccolte;
- un limite massimo di step (MAX\_RESEARCH\_STEPS = 10): un totale di dieci passi di ricerca, raggiunti i quali si passa comunque alla stesura, evitando che il modello chiami ricorsivamente tool e vada in overthinking.
- il blocco delle chiamate ripetute identiche: il nodo dei tool calcola una "firma" (nome dello strumento più argomenti) di ogni chiamata e la registra nello stato (done\_tool\_calls); se il modello richiama lo stesso strumento con gli stessi argomenti, la chiamata viene bloccata e il modello invitato a cambiare strategia. Nasce da un caso reale in cui un modello richiama tre volte di fila lo stesso strumento con la stessa query fallimentare;

- la semplificazione deterministica delle query lato server: le query di ricerca vengono accorciate (tagliate alla prima virgola e ad un massimo di parole), perché i modelli piccoli tendono a generare query lunghe e "ricche" che peggiorano i risultati;
- la normalizzazione dei nomi dei parametri dei tool, perché un modello piccolo a volte sbaglia il nome di un argomento;
- un nodo resiliente che tollera il fallimento di uno strumento (nome inesistente, argomenti malformati) e restituisce un messaggio istruttivo invece di bloccare l'intero processo;
- un guardrail di verifica delle fonti: se l'agente sta per scrivere senza aver raccolto alcuna fonte, viene forzata una ricerca web prima della stesura (una sola volta, tramite il flag `forced_web_search`).

Il ciclo è inoltre intrinsecamente limitato dalla combinazione del tetto sulle ricerche web e del blocco delle ripetizioni, oltre al limite di ricorsione del grafo come ultima rete di sicurezza.

## Motivazioni delle scelte implementative e limiti incontrati

La filosofia dei guardrail è il punto più importante di questo capitolo: la verifica delle fonti, i limiti di ricerca, il blocco delle ripetizioni, la clamp del numero di post, avvengono ma non vanno mai forzare la scelta generale dei tool. Forzare quale strumento usare snaturerebbe il paradigma ReAct, trasformando l'agente in una procedura rigida. La scelta degli strumenti resta quindi al modello; i guardrail intervengono solo in punti specifici e giustificati, dove la necessità è certa (per esempio: prima di scrivere bisogna avere almeno una fonte, oppure: non ha senso ripetere una chiamata identica che è già fallita).

Lo stesso vale per il formato del ragionamento: imporre la struttura Thought / Action serve alla trasparenza e alla tracciabilità, ma il contenuto delle decisioni resta del modello.

Tra le alternative studiate per migliorare l'uso dei tool da parte dell'agente è nata l'idea di forzare deterministicamente la sequenza di tool prevista dal piano: scartata, perché snatura ReAct e perché il piano è espresso in linguaggio naturale, quindi fragile da interpretare in modo affidabile. Successivamente, visto che uno dei requisiti era l'utilizzo del fine tuning, si poteva anche fare il fine-tuning di un selettore di tool dedicato, ma successivamente scartata come scelta per il costo in termini di dataset e di tempo. L'altra valida alternativa era usare un'API esterna in cloud per far scegliere i tool a un modello più capace: scartata perché contraddice il vincolo dei modelli locali e introdurrebbe costi e dipendenza dalla rete.

Durante il ciclo ReAct il modello sceglie tra cinque strumenti; più la lista è lunga, più la scelta diventa difficile per un modello di piccole dimensioni. Gli altri strumenti del sistema (analisi SEO, generazione della copertina, scrittura nel grafo) non sono scelti dal modello, ma invocati deterministicamente nella fase di pubblicazione, proprio per non appesantire la selezione.

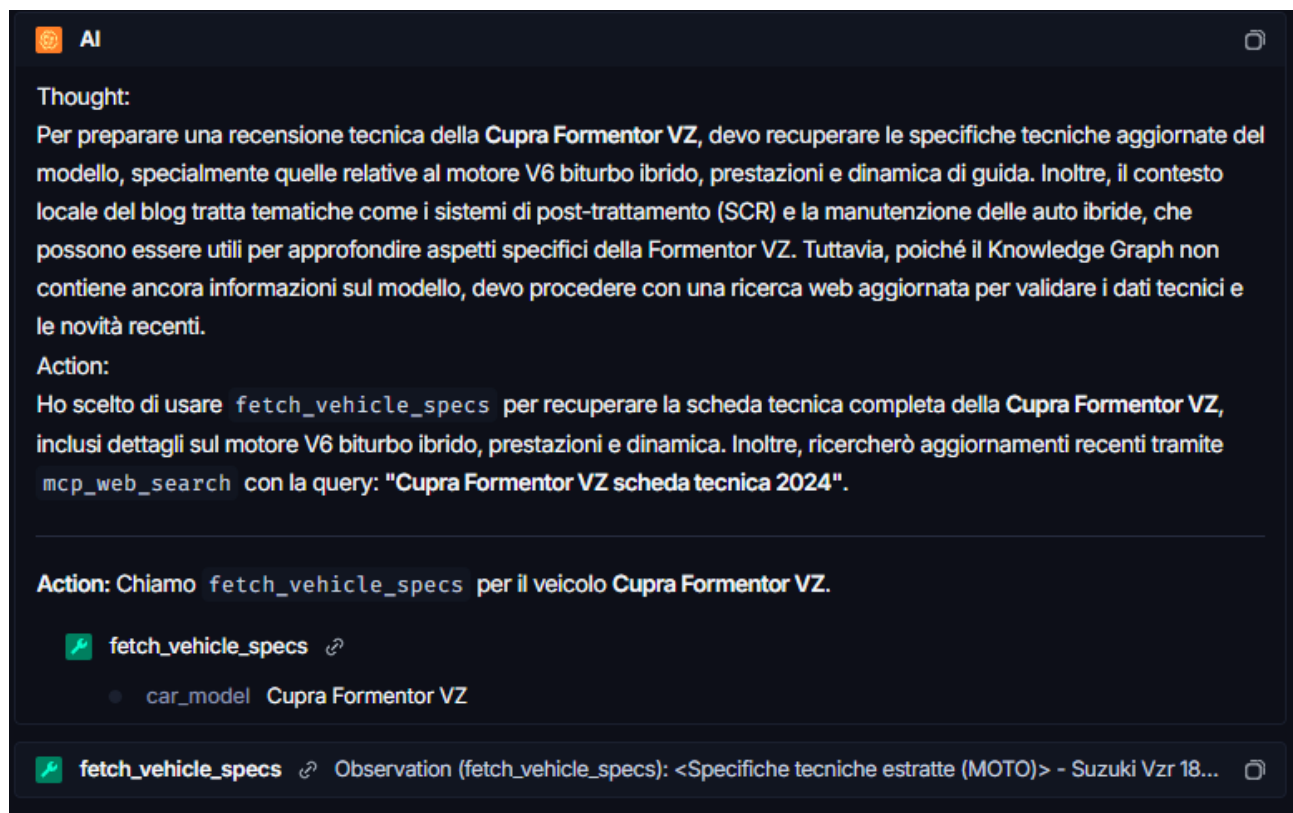
La soluzione finale adottata infatti, è stata quella di impiegare un modello più piccolo, come Ministral 3 da 3B, invece che la stessa versione ma da 8B, assegnandogli una maggiore



“context token window” in modo tale che il modellino non perdesse mai il contesto di esecuzione, da un nodo all’altro. Nonostante ciò, si è comunque osservato che il modello a volte pianifica correttamente gli strumenti nel suo ragionamento, ma poi ne salta qualcuno in fase di esecuzione. Inoltre, il formato ReAct non è sempre testualmente perfetto. Il modello esprime l'azione tramite il function-calling nativo di LangChain: la riga "Action:" non sempre compare come testo letterale, anche se il ciclo (pianificazione → chiamata → osservazione) avviene comunque ed è registrato nel reasoning\_trace. Imporre il formato esplicito ha richiesto un'attenta calibrazione del prompt di avvio (un'istruzione troppo generica aveva, in una fase intermedia, fatto degenerare il ragionamento sul solo Thought).

Senza i guardrail legato al blocco delle chiamate ripetute sui tool, il modello tendeva a ripetere la stessa chiamata fallimentare; il blocco delle ripetizioni risolve il caso, ma è il sintomo di una pianificazione non sempre lucida da parte di un modello piccolo. Vale lo stesso discorso per il limite sulle ricerche web: se le ricerche precedenti sono state spese su query poco efficaci, l'agente si trova comunque con fonti poco affidabili e tende comunque ad utilizzarle anche se imprecise.

Di seguito vengono mostrati tre screenshot presi da LangSmith, in cui possiamo osservare il ciclo ReAct nel research\_agent di una run; ed altri due screen in cui possiamo vedere la waterfall di una run specifica, riguardo il confronto tra due moto.





AI

Thought:

Il risultato ottenuto da `fetch_vehicle_specs` non corrisponde al modello **Cupra Formentor VZ** (che è un ibrido plug-in con motore V6 biturbo), ma solo a una versione storica o custom come la Suzuki VZR 1783. Quindi, il tool ha restituito dati generici e non specifici per il veicolo richiesto.

Dobbiamo quindi ricercare nuovamente le informazioni aggiornate sul **modello ibrido della Formentor VZ** tramite una ricerca web mirata. Inoltre, poiché il contesto locale del blog tratta la manutenzione di auto ibride e i sistemi di post-trattamento (SCR), sarà utile integrare queste conoscenze per approfondire aspetti tecnici specifici.

Action:

Ho scelto di usare `mcp_web_search` con la query: "**Cupra Formentor VZ scheda tecnica 2024 motore V6 biturbo ibrido**" per recuperare dati aggiornati e specifici sul modello. Inoltre, recherò eventuali novità recenti o test tecnici pubblicati su testate specializzate.

Action: Chiamo `mcp_web_search` con la query: "**Cupra Formentor VZ scheda tecnica 2024 motore V6 biturbo ibrido**".

 `mcp_web_search` 

- query `Cupra Formentor VZ scheda tecnica 2024 motore V6 biturbo ibrido`

 `mcp_web_search`  Observation (`mcp_web_search`): Risultati della ricerca web: FONTE 1: Titolo: Cupra Formentor...

AI

Thought:

Le ricerche precedenti non hanno trovato informazioni specifiche sul **motore V6 biturbo ibrido** della Cupra Formentor, né su eventuali dettagli tecnici aggiornati per il modello ibrido plug-in (1.5 TSI e-Hybrid). Poiché il Knowledge Graph contiene riferimenti ai **sistemi di post-trattamento (SCR)** e alla **manutenzione delle auto ibride**, mi concentro ora su:

- Verificare se la Formentor VZ sia un modello specifico legato a un motore V6 biturbo ibrido o se si tratti di una variante non ancora ufficialmente confermata.
- Ricercare informazioni più dettagliate sul **motore ibrido** (es. coppia massima, regimi di rotazione, sistema di recupero energetico) e sulle prestazioni dinamiche.

Poiché il tool `fetch_vehicle_specs` non ha restituito dati specifici per la Formentor VZ, procederò con una ricerca più mirata su fonti specializzate, utilizzando parole chiave che includano **motori ibridi**, **V6 biturbo** e **Cupra Formentor**. Inoltre, verificherò se esistono test tecnici o prove di guida pubblicati recentemente sul modello.

Action:

Ho scelto di usare `mcp_web_search` con la query: "**Cupra Formentor VZ motore V6 biturbo ibrido prestazioni 2024**" per recuperare eventuali dettagli specifici su un motore V6 biturbo ibrido nel contesto della Formentor. Se non risulta nulla, recherò ulteriormente con parole chiave correlate al sistema ibrido e alle innovazioni tecniche.

Action: Chiamo `mcp_web_search` con la query: "**Cupra Formentor VZ motore ibrido prestazioni 2024**".

 `mcp_web_search` 

- query `Cupra Formentor VZ motore ibrido prestazioni 2024`

 `mcp_web_search` 

Limite di 2 ricerche web raggiunto per questo post. Usa le informazioni già raccolte (fonti locali e web) per scrivere la bozza, senza altre ricerche.

4 LangGraph 7:15 PM

🕒 298.17s 📄 40.2K

- editorial\_review\_node 1.26s
  - kg\_recent\_posts 0.62s
- research\_agent 11.51s 📄 4.9K
  - kg\_topic\_context 1.44s
  - kg\_related\_topics 1.36s
  - rag\_local\_retrieval 0.05s
  - ChatOllama minstral-3:3b 8.64s 📄 4.9K
  - route\_after\_research 0.00s
- tools 124.81s 📄 5.8K
  - compare\_vehicles 124.81s 📄 5.8K
    - mcp\_web\_search 46.91s
    - ChatOllama minstral-3:3b 7.30s 📄 2.5K
    - mcp\_web\_search 54.24s
    - ChatOllama minstral-3:3b 7.23s 📄 2.2K
    - ChatOllama llama3.2:1b\_fine\_tuned 9.05s 📄 1.1K
  - grade\_documents 0.00s
- research\_agent 16.03s 📄 5.6K
  - ChatOllama minstral-3:3b 16.02s 📄 5.6K
  - route\_after\_research 0.00s
- tools 64.72s 📄 2.8K
  - mcp\_web\_search 58.34s
  - grade\_documents 6.39s 📄 2.8K

tools 64.72s 📄 2.8K

- mcp\_web\_search 58.34s
- grade\_documents 6.39s 📄 2.8K
  - ChatOllama minstral-3:3b 6.38s 📄 2.8K
  - PydanticOutputParser 0.00s
- research\_agent 34.73s 📄 9.2K
  - ChatOllama minstral-3:3b 34.73s 📄 9.2K
  - route\_after\_research 0.00s
- drafting\_node 45.09s 📄 11.9K
  - kg\_topic\_context 1.43s
  - ChatOllama minstral-3:3b 43.66s 📄 11.9K
- review\_node 0.00s

5 LangGraph 7:22 PM

🕒 12.37s 📄 1.6K

# Human-in-the-Loop (HITL)

## Introduzione all'HITL

L'**Human-in-the-Loop** (letteralmente "umano nel ciclo") è un approccio in cui un sistema automatico o di intelligenza artificiale mantiene una persona nel processo decisionale: l'AI svolge il lavoro, ma in punti chiave un essere umano rivede e approva prima di procedere. Le ragioni sono il controllo, la qualità, la sicurezza e la responsabilità: per contenuti che verranno pubblicati, non è prudente lasciare che un agente agisca in piena autonomia. In termini di agente, l'HITL si traduce in un punto di pausa (nel nostro caso un'interrupt) in cui il sistema si ferma e attende l'intervento umano prima di continuare.

Tecnicamente, in LangGraph questo si realizza con il meccanismo `interrupt()`, reso possibile dal checkpointing: quando un nodo chiama `interrupt()`, il grafo salva lo stato e si sospende; alla risposta dell'utente, l'esecuzione riprende esattamente da quel punto tramite un comando di `resume`. È utile sapere che, nell'osservabilità di LangSmith, questa pausa appare come un'eccezione (`GraphInterrupt`): non è un malfunzionamento, ma la rappresentazione interna della sospensione in attesa dell'intervento umano.

Ogni pausa è descritta da una richiesta strutturata con tre campi:

- `action_request`: il tipo di azione richiesta (per esempio `review_post_draft`) ed eventuali argomenti;
- `config`: quali risposte sono ammesse in quel punto, tramite quattro flag: `allow_accept` (approva così com'è), `allow_respond` (rispondi con un testo libero), `allow_ignore` (ignora/salta), `allow_edit` (modifica diretta del contenuto);
- `description`: il contenuto da mostrare all'utente (la bozza, le proposte, i suggerimenti).

La risposta dell'utente torna al grafo come un *resume* con un campo `type` che vale `accept`, `response`, `ignore` o `edit`, più eventuali argomenti. Ogni nodo-gate legge il `type` e instrada di conseguenza. Avere un protocollo unico per tutti i gate rende il sistema uniforme e facilmente estendibile a un'interfaccia grafica (come Agent Inbox) al posto del terminale.

## I cinque punti di Human-in-the-Loop

Le specifiche richiedono almeno un punto di controllo umano, ovvero nella revisione della bozza prima della pubblicazione. Il progetto ne implementa cinque, perché l'obiettivo è dare al redattore il controllo sull'intera strategia editoriale, non solo sul testo finale. Quattro dei cinque gate sono quindi estensioni oltre il requisito minimo.

Un principio attraversa tutti i gate: la risposta dell'utente è interpretata da parser deterministici, non dal modello. Le decisioni editoriali (quali proposte scrivere, quale tema scegliere, se continuare) sono troppo importanti per essere affidate all'interpretazione di un modello piccolo, sbagliava sistematicamente comandi come "scarta il 3".

**Chiarimento** (clarify\_request, Fase 0). All'arrivo della richiesta, se questa è troppo vaga, l'agente si ferma e chiede di precisare. L'intercettazione delle richieste generiche è affidata in prima battuta a una rete di sicurezza deterministica (riconosce frasi come "scrivimi qualcosa", "parlami di auto") affiancata dal giudizio del modello; per non avvitarsi, il chiarimento può scattare al massimo due volte. Risposte ammesse: testo libero (la precisazione) o "ignora" (si procede comunque). Se la richiesta è già chiara, il gate non interrompe.

**Scelta dai suggerimenti** (choose\_suggestion, Fase 2b). Nel ramo "di cosa parliamo oggi?", dopo aver presentato proposte in sospeso, calendario e notizie RSS, l'agente chiede se l'utente vuole scriverne uno. Un parser deterministico interpreta la scelta in più forme: per lista e numero ("la proposta 1", "il calendario 2", "la notizia 3"), per numero "nudo" (con una priorità coerente con l'ordine di presentazione), per corrispondenza sul titolo ("quello sulla Porsche") o come tema libero. Se l'utente sceglie un tema, il flusso rientra nella scrittura ripartendo dal brief; altrimenti si chiude lasciando i suggerimenti come risposta.

**Revisione del piano editoriale** (review\_editorial\_plan, Fase 2a). L'agente presenta le proposte di post e attende le decisioni dell'utente. È il gate con il parser più ricco (\_parse\_editorial\_decision): in una sola frase l'utente può combinare più comandi — scrivere alcune proposte, modificarne altre con istruzioni ("il 2 rendilo un confronto con la BMW Serie 3"), scartarne altre ancora ("scarta il 3"), oppure chiedere nuove proposte ("proponi nuove") quando ne restano meno di quante richieste. La modifica rigenera solo la proposta indicata, lasciando le altre invariate. Le proposte confermate vengono salvate nel Knowledge Graph come backlog prima di iniziare a scrivere.

**Revisione della bozza** (review\_post\_draft, Fase 5). È il gate richiesto dalle specifiche. Il grafo si ferma e presenta la bozza appena scritta; l'utente può approvare (accept: la bozza viene pubblicata e salvata nel grafo), modificare (response: un feedback testuale che innesca la rigenerazione) o scartare (ignore: si termina senza salvare). Un dettaglio di design rilevante riguarda la modifica: il sistema distingue due tipi di richiesta. Una modifica puramente testuale ("rendi il tono più informale", "accorcia l'introduzione") non richiede nuove informazioni e torna direttamente al nodo di stesura; una modifica che richiede nuovi dati ("aggiungi un confronto con un altro modello") attiva invece un nodo di ricerca mirata, che raccoglie le informazioni mancanti prima di rigenerare. Distinguere i due casi evita ricerche inutili quando serve solo riscrivere.

**Avanzamento multi-post** (continue\_writing, Fase 7). Quando erano stati pianificati più post, dopo la pubblicazione di ciascuno l'agente chiede come proseguire: continuare con il successivo, sceglierne uno in particolare, o fermarsi. Tra un post e l'altro lo stato viene ripulito (messaggi, contatori, fonti) mantenendo la coda dei post selezionati.

In sintesi: il Gate 4 soddisfa il requisito; i Gate 1, 2, 3 e 5 estendono il controllo umano a tutte le decisioni editoriali rilevanti — chiarire l'intento, scegliere cosa scrivere, approvare il piano, decidere se proseguire.

## Motivazioni delle scelte implementative e limiti incontrati

L'HITL è stato inserito perché pubblicare contenuti in modo completamente autonomo sarebbe rischioso: il redattore deve mantenere il controllo editoriale. L'agente assiste e propone, ma è l'essere umano a decidere.

La scelta di portare l'HITL oltre il singolo gate richiesto nasce da una visione del prodotto: il valore di un assistente editoriale non sta solo nello scrivere un buon testo, ma nell'accompagnare il redattore lungo tutta la strategia come: capire cosa intende, proporgli un piano, lasciargli scegliere i temi, farsi approvare le bozze. Ogni gate corrisponde a una decisione editoriale reale che una persona prende davvero.

L'uso di parser deterministici in tutti i gate è la seconda motivazione di fondo, ed è coerente con la filosofia generale del sistema: la certezza dell'interpretazione, dove le conseguenze sono importanti, non va affidata a un modello.

La scelta di usare i parser non copre tutti casi previsti. I parser deterministici sono robusti sui modi d'uso attesi (testati su molti casi), ma una formulazione molto inusuale potrebbe non essere interpretata correttamente; in quei casi il gate ripropone la richiesta invece di agire a caso.

Di seguito vengono riportati due screen presi dalla CLI, in cui vengono mostrati due gate HITL, precisamente il primo relativo alla fase 2a e alla fase 5.

```
# Proposte editoriali
```

```
1. [review] [Review] Recensione tecnica della Cupra Formentor VZ: prestazioni, dinamica e innovazione del motore V6 biturbo ibrido
```

```
    Motivazione: Risposta diretta alla richiesta dell'utente. Il post affronta il veicolo specifico richiesto (Cupra Formentor VZ), con un focus tecnico-pratico sulla sua dinamica di guida, prestazioni e innovazione del motore V6 biturbo ibrido. Questo approfondimento è coerente con il gap analizzato sul blog: i motori V6 biturbo e le sportive ibride sono argomenti mai trattati in dettaglio prima, offrendo un valore aggiunto rispetto ai post già pubblicati.
```

```
Cosa vuoi fare? Esempi:
```

```
- "scrivi 1 e 3"  
- "scrivi 1; il 2 rendilo un confronto con la BMW Serie 3; scarta il 3"  
- "scrivili tutti" - "annulla"
```

```
-----  
Come vuoi procedere?
```

```
1) Approva -> salva il post nel Knowledge Graph  
2) Modifica -> richiedi cambiamenti e rigenera la bozza  
3) Scarta -> annulla senza salvare
```

```
Scelta [1/2/3]: █
```

## Model Context Protocol (MCP)

### Introduzione all'MCP

Il Model Context Protocol (MCP) è uno standard aperto (introdotto da Anthropic) per collegare le applicazioni di intelligenza artificiale a strumenti e fonti di dati esterni in modo uniforme.

Per capire il problema che risolve, si immagini di dover collegare diverse applicazioni AI a diversi servizi esterni: senza uno standard, ogni integrazione va costruita su misura, e il numero di collegamenti cresce in modo moltiplicativo (il cosiddetto problema "M×N"). L'MCP standardizza queste connessioni con un modello client-server: gli strumenti e i dati vengono esposti da server MCP, e le applicazioni AI (i client) vi si collegano attraverso un protocollo comune.

L'architettura prevede quindi un server che espone delle capacità (strumenti, risorse) e un client che si connette e le utilizza, comunicando tramite un protocollo definito (per esempio su HTTP). Il vantaggio è il disaccoppiamento: il servizio esterno può evolvere in modo indipendente dall'applicazione che lo usa.

Nel contesto di questo progetto il termine "MCP" può essere inteso in due modi, ed è utile distinguerli perché il progetto li copre entrambi:

- Gestione del contesto in senso lato (la memoria di lavoro dell'agente): nel sistema è realizzata dallo stato condiviso che attraversa il grafo LangGraph, il quale tiene insieme richiesta, ragionamento, fonti, bozza e contatori lungo tutte le fasi, con persistenza tramite checkpointing.
- Il Model Context Protocol vero e proprio: il protocollo standardizzato client-server descritto sopra, che il progetto implementa concretamente realizzando un **server MCP per la ricerca web**. Questa parte è un'estensione che va oltre la semplice gestione del contesto.

## Implementazione dell'MCP nel progetto

Nel progetto la ricerca web è incapsulata come un server MCP indipendente. Il server è realizzato con FastMCP (il framework di riferimento per costruire server MCP in Python): viene istanziato come FastMCP("AutomotiveSearchServer", host="127.0.0.1", port=8765) ed espone una sola capacità, lo strumento `search_and_summarize`, dichiarato con il decoratore `@mcp_server.tool()`. Il server viene avviato in modalità `transport="streamable-http"`, cioè comunica via HTTP.

Il punto importante è che il server non si limita a inoltrare la ricerca: incapsula un'intera pipeline di qualità che gira nel suo processo. Quando riceve una query, il server la semplifica in modo deterministico (taglio alla prima virgola e a un numero massimo di parole, perché i modelli piccoli tendono a produrre query troppo lunghe), poi interroga Tavily in profondità (`search_depth` avanzato, fino a cinque risultati) limitandosi a una whitelist di testate automotive affidabili, successivamente deduplica e filtra i risultati scartando il rumore di pagina, e infine riassume ogni fonte singolarmente con il modello Ministral 3B (un prompt dedicato che conserva i dati tecnici e taglia pubblicità e contenuti inutili). Il risultato restituito al client è un testo già pulito e strutturato per fonte (titolo, riassunto, URL), nel formato "FONTE 1 / FONTE 2 / ...".

Dal lato dell'agente, il client è costruito con la libreria langchain-mcp-adapters (MultiServerMCPClient), configurato per raggiungere il server all'indirizzo `http://127.0.0.1:8765/mcp` con transport `streamable_http`. Il client interroga il server per la lista degli strumenti esposti, individua `search_and_summarize` e lo invoca. Poiché il grafo dell'agente è sincrono mentre la comunicazione MCP è asincrona, la chiamata viene eseguita in un event loop dedicato all'interno di un thread separato, con un timeout di 90 secondi e una gestione degli errori che, in caso di server spento o di mancata risposta, restituisce un messaggio chiaro e lascia proseguire l'agente senza bloccarsi. All'agente tutto questo è esposto semplicemente come lo strumento `mcp_web_search`.

Lo stesso strumento `mcp_web_search` è riusato anche dal tool di confronto tra veicoli (`compare_vehicles`), che per recuperare i dati di ciascun veicolo passa per il server MCP: così la ricerca e la sintesi delle fonti avvengono sempre nel processo del server, in modo uniforme, e non vengono duplicate altrove.

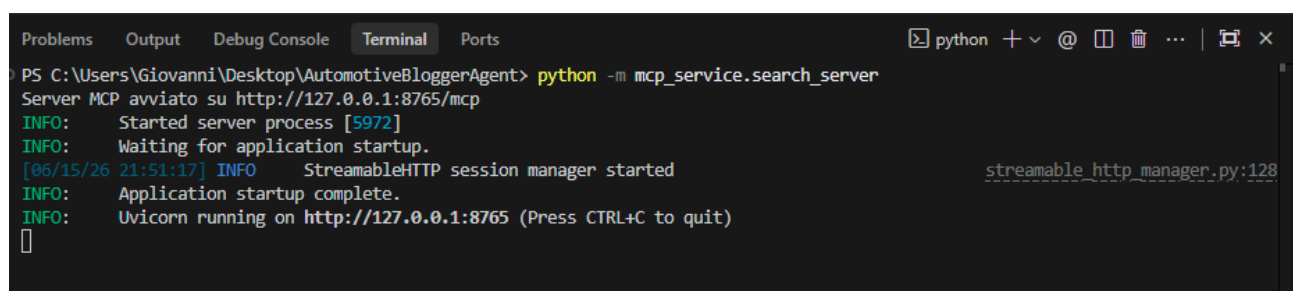
Da un punto di vista pratico, questo significa che per l'esecuzione servono due processi: in un terminale si avvia il server MCP (`python -m mcp_service.search_server`), in un altro si lancia l'agente, che vi si connette come client.

## Motivazioni delle scelte implementative e limiti incontrati

L'MCP è stato adottato per dimostrare il pattern di integrazione standardizzata e per disaccoppiare la ricerca web dal resto del sistema, incapsulandola come un microservizio autonomo. È stato applicato proprio alla ricerca web perché è una capacità esterna ben circoscritta, che si presta naturalmente al modello a servizio. Questo si è rivelato importante anche per il tool di confronto, che altrimenti avrebbe affollato il processo principale con molte chiamate di sintesi. Tra le alternative implementative vi era l'utilizzo di transport `stdio`, ma che nel nostro caso ha dato problemi pratici, non permettendo il corretto funzionamento del server e dunque abbandonato. Vi era inoltre l'idea di adottare l'MCP ad un altro componente fondamentale del progetto, il KG, ma ciò non è stato fatto (PERCHÉ?).

Concludendo con i limiti incontrati, sicuramente il primo riguarda la dipendenza dal server locale, nel caso in cui il server MCP non è attivo (o non funziona come nel caso di transport `stdio`), lo strumento di ricerca web fallisce e non viene eseguito, perdendo la fonte di grounding più importante per l'intero sistema.

Di seguito è riportato uno screen della CLI in cui viene eseguito processo server dell'MCP.



```
Problems  Output  Debug Console  Terminal  Ports
PS C:\Users\Giovanni\Desktop\AutomotiveBloggerAgent> python -m mcp_service.search_server
Server MCP avviato su http://127.0.0.1:8765/mcp
INFO: Started server process [5972]
INFO: Waiting for application startup.
[06/15/26 21:51:17] INFO StreamableHTTP session manager started streamable_http_manager.py:128
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8765 (Press CTRL+C to quit)
```



# Architettura multi-modello

## La sfida dei modelli locale e le scelte adottate

Una scelta di fondo del progetto è eseguire tutti i modelli linguistici in locale, tramite Ollama, senza ricorrere ad API in cloud. Questo comporta un vincolo importante: la quantità di memoria della scheda grafica (VRAM) è limitata, e i modelli che vi entrano sono di dimensioni contenute. I modelli piccoli, però, hanno comportamenti meno affidabili, in particolare nella scelta degli strumenti, e una qualità di scrittura variabile. La sfida è stata ottenere un sistema affidabile e di buona qualità entro questi limiti.

Invece di scegliere un modello a priori, la decisione è stata presa sperimentalmente, confrontando più modelli sullo stesso compito. Il percorso ha attraversato diversi candidati: modelli della famiglia Gemma, Qwen (scartato perché troppo lento sull'hardware a disposizione), Phi-4 Mini, Granite e infine Ministral 3B, oltre ad alcuni modelli di taglia maggiore (7-8 miliardi di parametri). Da questo confronto sono emersi due risultati significativi.

Il primo riguarda il tool-calling: modelli diversi scelgono gli strumenti con affidabilità molto diversa, e non è solo una questione di dimensione. Un modello più piccolo ma più recente e addestrato bene sul tool-calling può comportarsi meglio di uno più grande ma più generalista.

Il secondo, e più importante, riguarda il rapporto tra dimensione del modello e finestra di contesto a parità di VRAM disponibile. Su una scheda con memoria limitata esiste un compromesso diretto: un modello più grande occupa più VRAM e lascia meno spazio per il contesto, mentre un modello più piccolo lascia spazio per una finestra di contesto più ampia. Le prime configurazioni impiegavano modelli di taglia maggiore (8 miliardi di parametri, sia per Granite sia per Ministral) con una finestra di contesto fissata manualmente a 8192 token, il massimo consentito dalla VRAM. Il risultato era problematico su tre fronti: lentezza, frequente spillover nella RAM di sistema (quando la VRAM si saturava, parte del modello finiva nella memoria centrale), e una perdita di contesto che si traduceva in un comportamento concreto e osservabile, il Ministral da 8B spesso non usava i tool che aveva "ragionato" nella fase ReAct, ma ne sceglieva altri a caso, segno che il contesto ristretto troncava parte del prompt e il modello "vedeva" solo istruzioni parziali (un input di fatto castrato).

Passando al modello da 3 miliardi di parametri (Ministral 3B), più leggero, si è potuta ampliare di molto la finestra di contesto: nella configurazione attuale il modello principale e quello di stesura lavorano con 22.528 token di contesto, e il modello che riassume le fonti web addirittura con 22.528 token (un riassunto fedele non deve perdere informazioni). Il modello da 3B, a differenza di quello da 8B, usa quasi sempre i tool che ha effettivamente pianificato nel suo ragionamento, oltre a essere più veloce. Questo ha permesso di distinguere ciò che era un limite di configurazione, da ciò che è un limite intrinseco del modello.



## Architettura Ibrida

Il confronto sperimentale ha portato a un'architettura in cui ogni compito è affidato al modello più adatto, con una progressiva unificazione dove i test mostravano che un solo modello bastava:

- Ministral 3B fa da modello principale ("cervello" e "penna" insieme): si occupa di scoping, pianificazione, scelta degli strumenti nel ciclo ReAct, valutazione delle fonti (Self-RAG), stesura dell'articolo e aggiornamento del Knowledge Graph. Inizialmente la sintesi dei risultati web era affidata a un modello separato (Phi-4 Mini), ma un confronto diretto su testi reali ha mostrato che Ministral sintetizzava con pari o maggiore fedeltà ai dati ed era più veloce: si è quindi deciso di unificare anche questo compito su Ministral, eliminando un modello e riducendo gli scambi in VRAM.
- Llama 3.2 1B, specializzato tramite fine-tuning, si occupa esclusivamente del confronto strutturato tra veicoli, all'interno di una pipeline in cui Ministral raccoglie e sintetizza prima i dati dei due veicoli.

Sebbene il modello sia lo stesso (Ministral 3B) per quasi tutti i compiti, ogni ruolo è configurato con la temperatura adatta al suo scopo: 0.0 (deterministica) per i nodi che devono dare un output stabile e ripetibile (scoping, pianificazione, parsing, scelta dei tool), 0.6 per la stesura dell'articolo (serve un minimo di creatività), 0.2 per il riassunto delle fonti (deve restare fedele ai dati, con pochissima libertà). Il giudice fine-tuned lavora a 0.1, quasi deterministico, perché il verdetto deve essere coerente.

Per gestire la memoria limitata, i modelli non in uso vengono scaricati dalla VRAM per fare spazio (Ollama serializza l'accesso alla GPU, caricando un modello per volta); il giudice fine-tuned, in particolare, viene invocato con `keep_alive=0`, cioè rilasciato subito dopo l'uso per restituire memoria a Ministral.

## Motivazioni delle scelte progettuali e limiti incontrati

I test mostravano che un solo modello era sufficiente, dunque si è preferito unificare per ridurre la complessità e gli scambi in VRAM. Tuttavia, tra le alternative percorribili, vi era la possibilità di utilizzare un modello più grande, usando meno contesto, che avrebbe portato ad un peggioramento nella scelta dei tool ed anche ad un rallentamento complessivo del sistema dovuto alla saturazione della VRAM. Vi è stata inoltre la possibilità di adottare un modello esterno in cloud (via API) come "cervello": un'alternativa che consisteva nell'affidare le decisioni (la scelta degli strumenti) a un modello esterno più capace via API, facendo poi eseguire i tool scelti ai modelli locali. Avrebbe migliorato l'affidabilità del tool-calling, ma è stata scartata perché contraddice il vincolo dei modelli interamente locali e introdurrebbe costi e dipendenza dalla rete. Contemporaneamente all'uso di un modello cloud per la tool-calling, si è esplorata la possibilità di usare un "tool-selector" locale fine tuned: ovvero sia un modellino anche da pochi parametri (da 1 a 3/4B circa) che sarebbe stato impiegato unicamente nella scelta dei tool; l'idea è stata successivamente scartata perché

l'addestramento di un modello locale con il fine di scegliere unicamente i tool, sarebbe stato un'inutile costo in termini di dataset e tempo, ed avrebbe portato un miglioramento marginale nella selezione dei tool. La soluzione definita infatti, è stata quella di adottare un modello più piccolo (la variante da 3B di Ministral 3) con una finestra di contesto molto elevata, in sostituzione di modelli più grande con finestre di contesto limitate.

L'intera esecuzione locale ha introdotto delle significative latenze, infatti far girare modelli su GPU "consumer" come una RTX 2070 ed una RTX 4060 mobile, comporta tempi di esecuzione dell'agente e stesura dell'articolo dell'ordine dei 8/10 minuti. Per una riduzione consistente delle latenze si sarebbe potuto adottare un modello cloud, tramite l'utilizzo di API, che avrebbe sicuramente migliorato l'agente in termini di latenze e performance, ma al tempo stesso avrebbe introdotto dei costi nello sviluppo ed esecuzione dell'intero progetto.

## Fine-Tuning

### Introduzione al Fine-Tuning

Il fine-tuning è una delle richieste centrali del progetto, l'obiettivo non è insegnare al modello l'italiano o cosa sia un'automobile, ma specializzarlo a produrre un formato di confronto preciso e ripetibile tra due veicoli, che il modello base conosce concettualmente ma non rispetta in modo affidabile. Il modello specializzato è quello che alimenta lo strumento `compare_vehicles`, soddisfacendo la richiesta che almeno uno dei tool aggiuntivi sia basato su un modello fine-tuned.

Un LLM viene creato in due fasi: il pre-training su enormi quantità di testo generico (impara grammatica, fatti, ragionamento) e il fine-tuning, un addestramento aggiuntivo su un dataset piccolo e specifico per specializzarlo su un compito. Si fonda sul transfer learning: la conoscenza generale viene riusata e adattata, invece di ripartire da zero. È vantaggioso perché richiede pochi dati e poca potenza di calcolo rispetto al pre-training.

Il full fine-tuning aggiorna tutti i parametri del modello: per un modello da 1B significa aggiornare un miliardo di numeri a ogni passo, con enorme consumo di memoria (pesi, gradienti e stati dell'ottimizzatore per ogni parametro). Il PEFT (Parameter-Efficient Fine-Tuning) aggiorna invece solo una piccola frazione dei parametri (spesso meno dell'1%), congelando il resto: l'idea è che per specializzare un modello non serve modificare tutto, bastano piccoli aggiustamenti mirati.

La tecnica PEFT usata è LoRA (Low-Rank Adaptation). Quando si fa fine-tuning, ogni matrice di pesi  $W_0$  viene modificata di una quantità  $\Delta W$ ; nel full fine-tuning  $\Delta W$  è grande quanto  $W_0$ . L'intuizione di LoRA è che  $\Delta W$  ha un rango intrinseco basso e può essere scomposto nel prodotto di due matrici molto più piccole,  $\Delta W = B \cdot A$ , dove con  $W_0$  di dimensioni  $d \times k$  si ha  $B$  di dimensioni  $d \times r$  e  $A$  di dimensioni  $r \times k$ , con il rango  $r$  molto più piccolo di  $d$  e  $k$ .

La tecnica usata nel progetto è un'evoluzione di LoRA: QLoRA. È LoRA con in più la quantizzazione a 4 bit del modello base. Quantizzare significa rappresentare i pesi con meno

bit (4 invece di 16/32), riducendo di circa quattro volte la memoria occupata dal modello base, al costo di una piccola perdita di precisione numerica. QLoRA usa il tipo dato NF4 (ottimizzato per pesi a distribuzione normale), la doppia quantizzazione (si quantizzano anche le costanti di quantizzazione) e calcola comunque in bfloat16 (la quantizzazione riguarda lo storage, non l'aritmetica). Il modello base quantizzato resta congelato; gli adattatori LoRA restano a 16 bit.

L'addestramento è un Supervised Fine-Tuning: per ogni input c'è la risposta corretta. Due concetti sono centrali. Il chat template è la formattazione con token speciali che Llama 3.2 si aspetta per distinguere i ruoli system/user/assistant. La perdita si calcola solo sulla risposta, non sul prompt, perché vogliamo che il modello impari a generare la risposta, non a ripetere la domanda.

## Sviluppo del training e implementazione nel progetto

Il modello di partenza è meta-llama/Llama-3.2-1B-Instruct, già pre-addestrato e istruito a seguire istruzioni. La dimensione è stata volutamente scelta abbastanza piccola in modo da addestrarla su Kaggle usando la configurazione delle 2 NVIDIA T4 da 16GB l'una.

Il dataset finale conta 274 esempi, ciascuno una tripletta di messaggi system/user/assistant (l'input sono i dati tecnici dei due veicoli, l'output il confronto strutturato atteso). Dopo un controllo d'integrità (ogni riga è JSON valido, ruoli nell'ordine corretto), il dataset è diviso in tre parti con dimensioni esatte e seed=42 per la riproducibilità: 174 training, 60 validation, 40 test. Il train addestra il modello, la validation serve a scegliere gli iperparametri e a monitorare l'overfitting, il test contiene veicoli mai visti in addestramento, così la valutazione misura la vera capacità di generalizzazione e non la memorizzazione. Prima dell'addestramento il notebook visualizza la composizione del dataset (auto/moto, esiti per categoria, lunghezza delle risposte) per verificarne il bilanciamento.

Per quanto riguarda la configurazione LoRA, si è scelto di usare un rango  $r=16$ , lora\_alpha=32 (cioè  $2r$ ), lora\_dropout=0.05 (regolarizzazione contro l'overfitting su dataset piccolo), bias="none", task\_type="CAUSAL\_LM", e target\_modules su tutte e sette le proiezioni di Llama: le quattro dell'attenzione (q\_proj, k\_proj, v\_proj, o\_proj) e le tre del blocco feed-forward (gate\_proj, up\_proj, down\_proj). Targettare tutte le proiezioni è la prassi moderna per l'SFT di LLM, perché distribuisce l'adattamento su tutto il modello. BitsAndBytesConfig con load\_in\_4bit=True, bnb\_4bit\_use\_double\_quant=True, bnb\_4bit\_quant\_type="nf4", bnb\_4bit\_compute\_dtype=torch.bfloat16. Dopo il caricamento si disattiva la cache (use\_cache=False, incompatibile con il gradient checkpointing in addestramento) e si chiama prepare\_model\_for\_kbit\_training, che riporta alcuni strati delicati a precisione più alta per stabilità.

L'addestramento usa SFTTrainer (libreria TRL), che applica automaticamente il chat template di Llama 3.2 e calcola la loss solo sui token dell'assistant, motivo per cui è preferito al Trainer generico. Il tokenizer imposta il pad\_token uguale all'eos\_token e padding\_side="right" (convenzione corretta per Llama con SFTTrainer).

I valori usati per l'addestramento finale sono:

| <i>Iperparametro</i>         | <i>Valore</i>      | <i>Motivazione</i>                                                         |
|------------------------------|--------------------|----------------------------------------------------------------------------|
| <i>Epoche</i>                | 5                  | Avendo una totalità di 174 esempi, ognuno è “visto” 5 volte                |
| <i>Batch per device</i>      | 2                  | Limite della VRAM delle T4                                                 |
| <i>Gradient accumulation</i> | 8                  | Batch effettivo $2 \times 8 = 16$                                          |
| <i>Learning Rate</i>         | 5e -4              | Scelto nella ricerca degli iperparametri, volutamente alto adatto per LoRA |
| <i>Scheduler</i>             | Cosine + warmup 5% | Sale lentamente e poi discende a coseno                                    |
| <i>Ottimizzazione</i>        | Paged_adamw_8bit   | Stati a 8 bit e paging per non sovraccaricare la VRAM                      |
| <i>Precisione</i>            | Bf16               | Coerente con il compute della quantizzazione                               |
| <i>Gradiend Checkpoiting</i> | Usato              | Trade-off tra calcolo e memoria                                            |
| <i>Eval Strategy</i>         | Per epoca          | Genera la validation loss per ogni epoca                                   |

L'addestramento traccia training e validation loss. La training loss parte intorno a 2,1 e scende a circa 0,48 alla quinta epoca; la validation loss finale si attesta intorno a 0,77. Il punto qualitativo è il segnale che cercavamo, infatti la training loss scende nettamente e la validation loss resta vicina alla training loss e non risale entro le 5 epoche. Con un dataset piccolo il rischio era l'overfitting; il fatto che la validation resti allineata dimostra che il modello ha imparato a generalizzare, non a memorizzare (in caso contrario, la training loss sarebbe crollata verso zero e la validation sarebbe schizzata in alto).

Prima del training definitivo si confrontano più configurazioni, variando un parametro alla volta a partire da una base, ricaricando un modello fresco per ognuna e addestrando su un sottoinsieme dei dati per velocità. Il confronto avviene sulla validation loss (mai sul test set). Le configurazioni testate:

| <i>Configurazione</i> | <i>Rango r</i> | <i>Learning Rate</i> | <i>Epoche</i> | <i>Validation loss</i> |
|-----------------------|----------------|----------------------|---------------|------------------------|
| <hr/>                 |                |                      |               |                        |

|   |    |      |   |       |
|---|----|------|---|-------|
| A | 16 | 2e-4 | 5 | 0,87  |
| B | 8  | 2e-4 | 5 | 0,93  |
| C | 16 | 5e-4 | 5 | 0,78  |
| D | 16 | 5e-4 | 7 | 0,782 |

Il learning rate è l'iperparametro con più impatto: alzarlo da 2e-4 a 5e-4, migliora più che cambiare il rango. Ridurre il rango peggiora, segno che servono abbastanza parametri di adattamento. La config D mostra perché 5 epoche sono il punto giusto: portando l'addestramento a 7 epoche, la validation loss tocca il minimo proprio intorno alla 5ª epoca ( $\approx 0,77$ ) e poi risale leggermente (circa 0,78 alle epoche 6-7) mentre la training loss continua a scendere fino a circa 0,40: il classico segnale di un lieve overfitting oltre la quinta epoca. Questo conferma sperimentalmente la scelta di fermarsi a 5 epoche, oltre le quali il modello inizia a memorizzare senza generalizzare meglio. Da qui la configurazione finale  $r=16$ ,  $lr=5e-4$ , 5 epoche.

Dopo l'addestramento si hanno due cose separate: il modello base e l'adapter LoRA (le matrici  $A$  e  $B$ ). Il merge calcola  $W = W_0 + BA$  per ogni matrice e produce un modello unico in cui l'adattamento è incorporato, usabile senza la libreria PEFT. Viene eseguito su CPU (`merge_and_unload()`) per non occupare la VRAM, già impegnata in fase di valutazione.

## Valutazione dei risultati del Fine Tuning

Inizialmente è stato fatto un confronto A/B: stesso modello, con e senza adapter quindi qualsiasi differenza è attribuibile al fine-tuning. Abbiamo usato un solo modello attivando/disattivando l'adapter (`disable_adapter_layers()` / `enable_adapter_layers()`), risparmiando memoria. Successivamente sono stati fatti dei test su dati nuovi: 40 casi con veicoli mai visti. Infine, si è valutato usando metriche di tipo diverso perché ciascuna cattura un aspetto diverso.

Le metriche standard sono state caricate con la libreria evaluate:

- BERTScore: misura la similarità di significato tramite embedding contestuali, riconoscendo le parafrasi. È la metrica più affidabile per la generazione, perché misura se il modello dice la cosa giusta, non se usa le parole identiche.

- ROUGE: misura la sovrapposizione lessicale (parole e sequenze in comune); complementare a BERTScore (se BERTScore dice *cosa* significa, ROUGE dice *quanto* coincide testualmente).
- BLEU: precisione degli n-grammi, con penalità di brevità; metrica severa, tende a valori bassi su testi strutturati diversamente.
- chrF: F-score sugli n-grammi di *caratteri*, robusta per lingue a morfologia ricca come l'italiano.
- Perplexity: misura quanto il modello trova "naturale" un testo (più bassa è meglio); misura di fluenza. Calcolata con una funzione custom che applica la formula al modello già in memoria, per poter usare il trucco enable/disable dell'adapter.

A queste si aggiungono tre analisi task-specifiche: la format compliance (controlla la presenza delle quattro categorie, dei vincitori, delle motivazioni e del verdetto), il Precision/Recall/F1 sui giudizi (ogni giudizio è trattato come classificazione a 3 classi: vince V1, vince V2, pareggio).

Di seguito vengono riportati i risultati ottenuti sul test set di 40 elementi:

| <b>Metrica</b>              | <b>Cosa misura</b>              | <b>Base</b> | <b>Fine-Tuned</b> | <b>Miglioramento</b> |
|-----------------------------|---------------------------------|-------------|-------------------|----------------------|
| <i>BERTScore F1</i>         | Significato (semantica)         | 0,6945      | 0,7714            | +11,07%              |
| <i>ROUGE-L</i>              | Sovrapposizione lessicale       | 0,2624      | 0,3736            | +42,36%              |
| <i>BLEU</i>                 | Precisione n-grammi             | 0,0927      | 0,2827            | +0,19                |
| <i>chrF</i>                 | Caratteri                       | 42,62       | 52,49             | +23,15%              |
| <i>Perplexity</i>           | Fluenza (più è bassa, meglio è) | 10,17       | 3,80              | -62,68%              |
| <i>Format Compliance</i>    | Conformità al formato richiesto | 88%         | 99%               | +11 punti            |
| <i>Accuracy dei giudizi</i> | Correttezza dei vincitori       | 36,3%       | 36,9%             | +0,6 punti           |
| <i>F1 dei giudizi</i>       | Qualità bilanciata dei giudizi  | 0,242       | 0,311             | +28,5%               |

Sul BLEU, la variazione percentuale (+202%) è gonfiata dal valore base quasi nullo (si divide per un numero piccolo): il riferimento corretto è il delta assoluto, +0,19 su scala 0-1. Sui giudizi, conviene citare l'F1 (+56,1%), che media equamente le tre classi, piuttosto che precision e recall singole, amplificate dal fatto che il base parte da zero sulla classe "pareggio".

Tutte le metriche concordano: il fine-tuning ha funzionato, con margini netti. I guadagni più forti sono dove ci si aspetta che il fine-tuning lavori meglio, la fluenza (perplexity -62,7%: il modello trova il formato molto più naturale) e l'aderenza al formato (portata al 99%). Un punto di onestà metodologica importante: anche il modello base è stato dotato del system prompt che descrive il formato; quindi, non parte "da zero" (raggiunge l'88% di format compliance); il fine-tuned vince comunque su tutte e otto le metriche, anche se sull'accuratezza dei giudizi il margine è minimo (come spiegato sotto), il che rende il confronto più severo e credibile.

Un risultato qualitativo merita attenzione: il guadagno del fine-tuning non sta tanto nella correttezza dei vincitori (l'accuracy è quasi identica, 36,3% contro 36,9%), quanto nella disciplina della risposta. Il modello base tende a dilungarsi e in molti casi non conclude il confronto: spesso omette i vincitori, le motivazioni o il Verdetto Finale (format compliance all'88%). Il modello fine-tuned, invece, rispetta il formato quasi sempre (99%) e soprattutto chiude il verdetto in modo molto più affidabile (categoria Verdetto azzeccata nel 72% dei casi contro il 61% del base). Analizzando l'accuracy per categoria, il fine-tuned è nettamente migliore su Sicurezza e Verdetto ma più debole su Consumi e Comfort, e i due effetti si compensano: per questo l'accuracy complessiva resta piatta, mentre tutto ciò che riguarda struttura, completezza e fluenza migliora nettamente. In sintesi, il fine-tuning ha insegnato al modello a rispettare il formato e a concludere il confronto, più che a giudicare "più giusto".

## Motivazioni delle scelte progettuali e limiti incontrati

Si è scelto QLoRA + SFTTrainer non perché siano "migliori" del full fine-tuning in assoluto, ma perché erano l'approccio più appropriato secondo noi. Un full fine-tuning sarebbe stato impossibile per un modello da oltre 1B di parametri su due T4 ed inoltre avrebbe riprodotto una copia quasi completa del modello invece che un'adapter da un centinaio di MB. L'alternativa era utilizzare il LoRA classico spiegato a lezione, che sarebbe sicuramente stata una scelta più che valida e anche più veloce, si è deciso però di usare QLoRA come scelta standard per esplorare una tecnica che viene utilizzata attualmente per modelli molto grandi che superano i 70B di parametri. Inoltre, il semplice prompt engineering non è bastato per permettere al modello base di generare una comparazione discorsiva perfetta, giustificando ancora di più la scelta del fine-tuning.

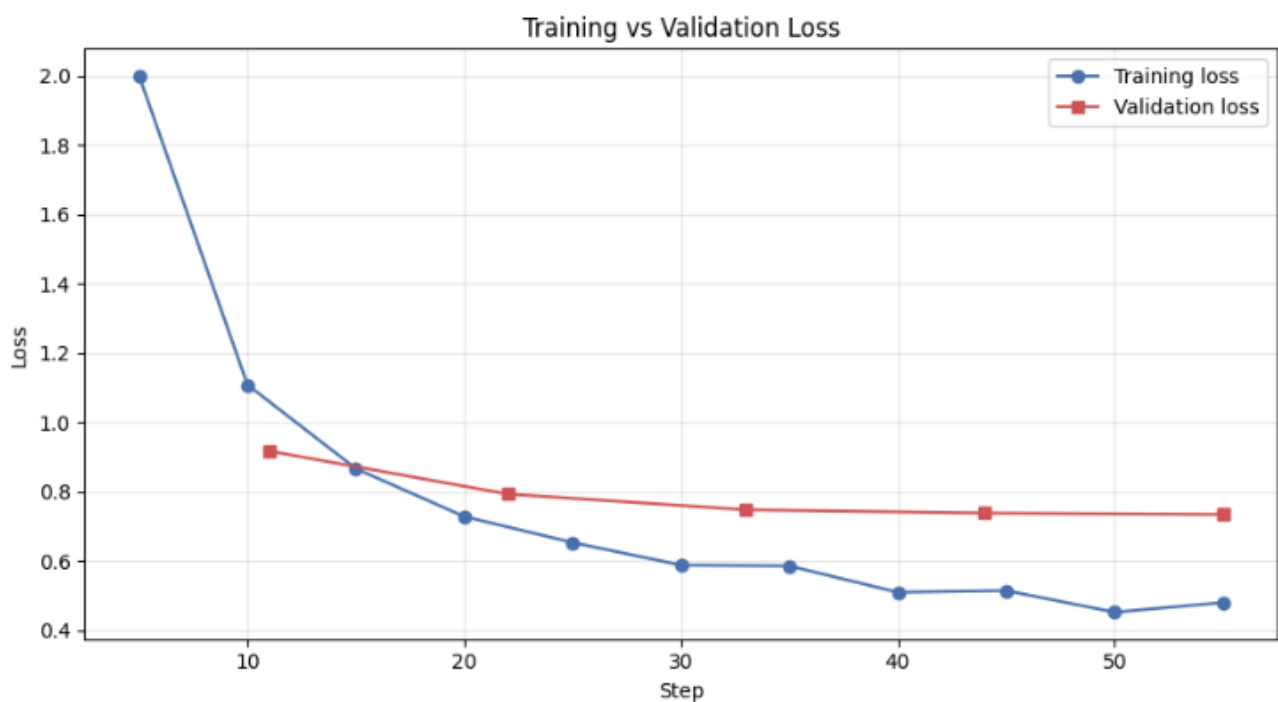
Tra i limiti principali facilmente riscontrabili è emerso chiaramente dal Precision/Recall/F1 sui giudizi, è il bias verso il pareggio: nei confronti più sfumati il modello tende a scegliere un pareggio invece di decretare un vincitore netto. Lo si vede dal recall altissimo (0,911) ma precision bassa (0,301) sulla classe PAREGGIO, e dal fatto che su V1 il fine-tuned alza la precision ma abbassa il recall (diventa più cauto). Il modello ha quindi imparato benissimo il formato e lo stile, ma la correttezza del giudizio nei casi vicini può ancora migliorare. È atteso con un dataset di queste dimensioni e indica lo sviluppo futuro: ampliare il dataset, in particolare con esempi che premiano il decretare un vincitore netto e bilancino meglio la frequenza dei pareggi.

Di seguito vengono riportati degli screen del notebook di addestramento eseguito su Kaggle, in cui possiamo vedere la scelta degli iperparametri, la curva di training e validation loss, il numero di parametri allenabili con LoRA vs quelli “congelati” e il riassunto con le metriche applicate al modello base e fine tuned.

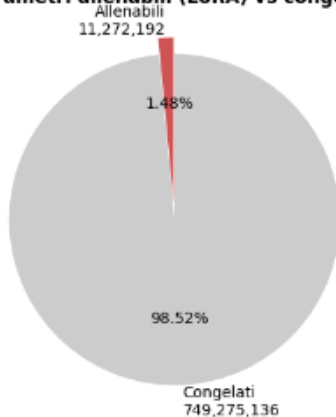
```
=====
CONFRONTO IPERPARAMETRI – ordinato per validation loss (crescente)
=====
```

| Config                 | Param. allenati | Train loss | Val loss | Val PPL |
|------------------------|-----------------|------------|----------|---------|
| C: r=16, lr=5e-4, ep=5 | 11,272,192      | 0.5581     | 0.7774   | 2.18    |
| D: r=16, lr=5e-4, ep=7 | 11,272,192      | 0.3995     | 0.7823   | 2.19    |
| A: r=16, lr=2e-4, ep=5 | 11,272,192      | 0.7693     | 0.8628   | 2.37    |
| B: r=8, lr=2e-4, ep=5  | 5,636,096       | 0.8771     | 0.9320   | 2.54    |

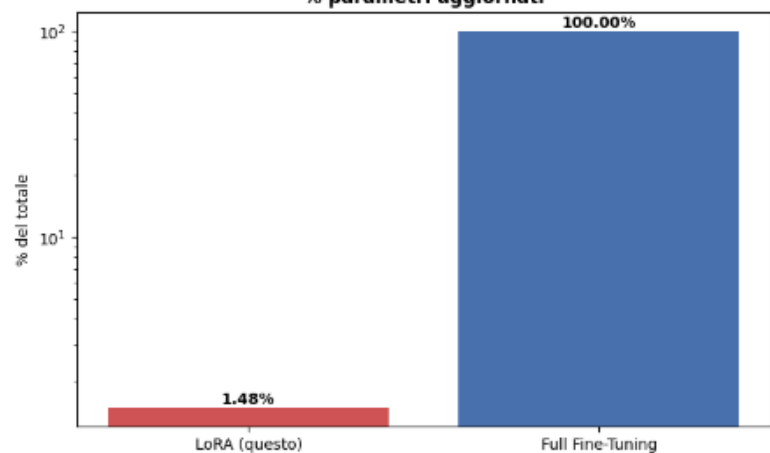
MIGLIOR CONFIG: C: r=16, lr=5e-4, ep=5  
 -> Usa r=16, lr=0.0005, epochs=5 per il training finale.



Parametri allenabili (LoRA) vs congelati



% parametri aggiornati





=====

ACCURACY PER CATEGORIA: quante volte il modello azzecca il vincitore

=====

| Categoria   | Fine-tuned     | Base           |
|-------------|----------------|----------------|
| Prestazioni | 8/36 (22%)     | 9/36 (25%)     |
| Consumi     | 4/36 (11%)     | 10/36 (28%)    |
| Sicurezza   | 18/36 (50%)    | 10/36 (28%)    |
| Comfort     | 10/35 (29%)    | 14/35 (40%)    |
| Verdetto    | 26/36 (72%)    | 22/36 (61%)    |
| TOTALE      | 66/179 (36.9%) | 65/179 (36.3%) |

=====

PRECISION / RECALL / F1 PER CLASSE – FINE-TUNED

=====

| Classe   | Precision | Recall | F1    | Casi |
|----------|-----------|--------|-------|------|
| V1       | 0.786     | 0.293  | 0.427 | 75   |
| V2       | 0.250     | 0.034  | 0.060 | 59   |
| PAREGGIO | 0.294     | 0.933  | 0.447 | 45   |

=====

PRECISION / RECALL / F1 PER CLASSE – BASE

=====

| Classe   | Precision | Recall | F1    | Casi |
|----------|-----------|--------|-------|------|
| V1       | 0.480     | 0.800  | 0.600 | 75   |
| V2       | 0.250     | 0.085  | 0.127 | 59   |
| PAREGGIO | 0.000     | 0.000  | 0.000 | 45   |

=====

RIEPILOGO FINALE SUI GIUDIZI – BASE vs FINE-TUNED

=====

| Metrica   | Base  | Fine-tuned | Miglioramento |
|-----------|-------|------------|---------------|
| Accuracy  | 0.363 | 0.369      | +1.5%         |
| Precision | 0.243 | 0.443      | +82.1%        |
| Recall    | 0.295 | 0.420      | +42.5%        |
| F1 macro  | 0.242 | 0.311      | +28.5%        |

=====

```

=====
FORMAT COMPLIANCE
=====
Fine-tuned : 99% compliance media
Caso 6: manca il verdetto
Base : 88% compliance media
Caso 6: manca i vincitori, le motivazioni
Caso 9: manca i vincitori, le motivazioni
Caso 14: manca i vincitori, le motivazioni
Caso 17: manca i vincitori, le motivazioni
Caso 19: manca i vincitori, le motivazioni
Caso 22: manca i vincitori, le motivazioni
Caso 24: manca le motivazioni
Caso 26: manca i vincitori, le motivazioni
Caso 30: manca le motivazioni
Caso 31: manca il verdetto
Caso 37: manca i vincitori
Caso 38: manca le motivazioni, il verdetto

```

```

=====
RIEPILOGO VALUTAZIONE: FINE-TUNED vs BASE (40 casi di test)
=====

```

| Metrica      | Base   | Fine-tuned | Miglioramento |
|--------------|--------|------------|---------------|
| BERTScore F1 | 0.6945 | 0.7714     | +11.07%       |
| ROUGE-L      | 0.2624 | 0.3736     | +42.36%       |
| BLEU         | 0.0927 | 0.2827     | +204.88%      |
| chrF         | 42.62  | 52.49      | +23.15%       |
| Perplexity   | 10.17  | 3.80       | -62.68%       |

```

=====

```

## Evaluation e Observability

### Introduzione all'osservabilità e valutazione

Costruire un agente non basta, bisogna infatti misurarne il comportamento in modo sistematico e ripetibile, non con prove aneddotiche. La valutazione del progetto segue un approccio preciso: si definiscono due dataset di prompt rappresentativi, si esegue l'agente su ciascun prompt, e si assegnano dei punteggi automatici a ogni esecuzione tramite degli evaluator. Tutto è tracciato su LangSmith, che registra ogni nodo, ogni chiamata ai modelli e ai tool, le latenze e i token, permettendo sia il punteggio aggregato sia l'ispezione della singola run.

Due evaluator usano un LLM come giudice (LLM-as-a-judge): un modello esterno legge l'output dell'agente e ne valuta qualità e fattualità. Come giudice principale si è scelto Gemini 3.1 Flash Lite, con fallback automatico su Ministral locale in caso di indisponibilità (Gemini ha un limite giornaliero di richieste). Usare un modello diverso da quelli dell'agente come giudice riduce il rischio di auto-compiacimento (un modello che valuta sé stesso tende a darsi

ragione). Gli altri due evaluator sono programmatici e deterministici: applicano controlli espliciti sull'output, senza ambiguità.

A livello di codice, la valutazione viene eseguita a partire dal file `evaluation/evaluate_blog.py` che esegue l'agente su due dataset, costruiti con script dedicati (`crea_dataset.py` e `crea_dataset_v2.py`) e caricati su LangSmith:

- Dataset 1 (5 prompt): un primo insieme di controllo, con casi mirati a coprire le diverse modalità dell'agente (scrittura di post, suggerimenti, confronti).
- Dataset 2 (25 prompt): un insieme più ampio e vario, che copre richieste di scrittura di post tecnici, recensioni di singoli modelli, confronti tra veicoli, richieste di attualità (saloni, novità) e richieste di sole proposte editoriali.

Su ogni esecuzione vengono calcolati quattro evaluator:

1. **Qualitative\_Score**: valuta la qualità redazionale del post (struttura, leggibilità, tono) su una scala da 1 a 5, normalizzata in 0-1. Se non viene generata alcuna bozza, vale 0.
2. **Source\_Grounding**: valuta l'ancoraggio alle fonti combinando in parti uguali due segnali: la presenza di citazioni esplicite nel testo (controllo programmatico, peso 0,5) e la fattualità del contenuto rispetto alle fonti raccolte (giudice LLM su scala 1-5, peso 0,5).
3. **Failure\_Cases**: verifica l'assenza di quattro modalità di fallimento, restituendo la frazione di controlli superati su quattro: F1 bozza vuota, F2 bozza priva di citazioni, F3 nessuna fonte raccolta dai tool, F4 esecuzione "appesa" senza output né interruzione di review.
4. **Tool\_Usage**: verifica l'uso corretto dei tool: parte da 1,0 e penalizza se non viene invocato alcun tool (0,3) o se nessuno dei tool usati è di ricerca/grounding (0,6); segnala inoltre (senza penalizzare) i casi in cui una richiesta di confronto non ha usato `compare_vehicles`, perché i dati possono comunque essere stati recuperati con la ricerca web senza allucinazioni.

Al termine, viene estratto da LangSmith un report di osservabilità (numero di esecuzioni, run con errori, tasso di errore, latenza, step medi per esecuzione).

Risultati ottenuti

Di seguito vengono riportati i punteggi medi dei quattro evaluator sui due dataset (scala 0-1, più alto è meglio):

| <i>Evaluator</i>         | <i>Dataset 1</i> | <i>Dataset 2</i> |
|--------------------------|------------------|------------------|
| <i>Tool_Usage</i>        | 1,00             | 1,00             |
| <i>Failure_Cases</i>     | 0,80             | 0,86             |
| <i>Source_Grounding</i>  | 0,60             | 0,71             |
| <i>Qualitative_Score</i> | 0,56             | 0,57             |

Successivamente vengono riportate le metriche di costi e prestazioni prese dalla dashboard di LangSmith:

| <b>Misura</b>                   | <b>Dataset 1</b> | <b>Dataset 2</b>  |
|---------------------------------|------------------|-------------------|
| <i>Latenza per prompt (P50)</i> | 156,8 s          | 138,5 s           |
| <i>Token totali</i>             | 135.880          | 1.024.927         |
| <i>Di cui input / output</i>    | 114.685 / 21.195 | 914.690 / 110.237 |
| <i>Run con errori</i>           | 0                | 0                 |
| <i>Tasso di errore</i>          | 0 %              | 0 %               |

Il dato più solido è l'uso dei tool: 1,00 su entrambi i dataset, cioè l'agente invoca sempre almeno un tool di ricerca/grounding appropriato, dimostrando coerenza con il lavoro fatto sul ReAct e sulle regole tassative di selezione dei tool. Anche l'assenza di errori (0% su 30 esecuzioni complessive) conferma la robustezza del grafo e dei meccanismi di fallback. Il grounding è discreto e migliora nel dataset più ampio (0,60 → 0,71).

Il punteggio di qualità (≈0,57) va letto con attenzione, perché è in parte un artefatto del modo in cui è costruita la valutazione, non solo un giudizio sulla scrittura. Il Qualitative\_Score (e il Failure\_Cases) presuppongono che ogni run produca un post completo; ma diversi prompt dei dataset sono richieste di sole proposte editoriali o si fermano correttamente al gate di approvazione umana (HITL), e quindi non hanno una bozza finale da valutare. Per questi casi il giudice assegna 0 alla qualità e il Failure\_Cases registra "bozza vuota": punteggi bassi che però non corrispondono a un fallimento reale, infatti l'agente ha fatto esattamente ciò che doveva (proporre, o attendere l'approvazione). Separando i due tipi di richiesta, i prompt di scrittura di post, mostrano Failure\_Cases vicino a 1,0 e qualità nettamente più alta (tipicamente 0,6-0,8), mentre sono i prompt di sole proposte/attesa a trascinare in basso la media. È un limite della valutazione (gli evaluator sono tarati sul post completo), più che dell'agente, e va dichiarato con onestà.

La latenza (≈2,3-2,6 minuti per prompt in mediana) è coerente con l'esecuzione di più modelli in locale su GPU di fascia mobile, come discusso in precedenza; il costo monetario è nullo, perché i modelli dell'agente girano in locale (l'unico servizio esterno è il giudice di valutazione).

Una seconda avvertenza riguarda proprio il giudice: in alcuni casi Gemini ha penalizzato bozze corrette perché la sua conoscenza dei modelli più recenti è datata, il che rende i punteggi di qualità e grounding pessimistici rispetto al valore reale di quei post.

## Failure cases documentati e limiti della valutazione

Oltre alle metriche aggregate, abbiamo isolato e documentato singoli casi di fallimento, perché capire dove e perché l'agente sbaglia è parte integrante della valutazione. Per ciascuno riportiamo sintomo, causa e, dove applicabile, la mitigazione adottata.

- FC1: Modello inventato. In una versione iniziale l'agente ha citato un'auto inesistente ("Alfa Romeo Giulia Quattrosotto"). Causa: il modello attingeva alla conoscenza interna senza ancoraggio. Mitigazione: contesto KG con i titoli reali e obbligo di usare i tool di ricerca.
- FC2: Fonti "sporche". Per la Mercedes-AMG A45 S il modello aveva prodotto dati errati (es. una cilindrata/potenza inventata) a partire da pagine web piene di rumore. Mitigazione: sintesi per-fonte lato server MCP, che pulisce le pagine prima di passarle all'agente.
- FC3: Modello allucinato. Sul prompt dell'Aprilia Tuono il modello ha generato specifiche non confermate; nel dataset 2 questo caso ottiene infatti punteggi di qualità e grounding bassi (qualità 0,40, grounding 0,50). Causa: modello recente con poche fonti affidabili disponibili.
- FC4: Tool esterno che restituisce il modello sbagliato. `fetch_vehicle_specs` ha talvolta restituito un veicolo diverso da quello richiesto (es. una Ford Mustang per una Peugeot 208), per via di un match impreciso dell'API esterna. Aspetto positivo: mostra anche la capacità di recupero del ciclo ReAct, che in alcuni casi riformula e cerca conferme.
- FC5: Giudice fine-tuned che omette il verdetto. Il modello da 1B che fa da giudice nel `compare_vehicles` talvolta non scriveva il "Verdetto Finale". Mitigazione: una chiamata di completamento mirata che aggiunge il verdetto se assente.
- FC6: Contenuto speculativo senza fonti. Sulla Maserati MCPura (modello molto recente) l'agente ha prodotto un testo in parte speculativo per scarsità di fonti; il caso compare nei dataset di valutazione.
- FC7: RAG che inietta un documento non pertinente. Il meccanismo di fallback del RAG, su temi senza documenti locali rilevanti, inietta talvolta il miglior chunk disponibile anche se marginale (es. `sistemi_post_trattamento` a distanza  $\approx 1,13$  per un confronto tra utilitarie). È il compromesso tra coprire i temi noti e non iniettare rumore.

Abbiamo già visto come, `Qualitative_Score` e `Failure_Cases` penalizzano i prompt di sole proposte o quelli che si fermano al gate umano, abbassando le medie senza che vi sia un fallimento reale. Una valutazione più equa userebbe evaluator differenziati per tipo di richiesta (scrittura vs proposta vs confronto).

Due dei quattro evaluator si appoggiano a Gemini 3.1 Flash Lite, ed essendo un modello piccolo ed economico la sua conoscenza dei veicoli molto recenti è incompleta: in più casi ha penalizzato come "false" o "inesistenti" informazioni in realtà corrette, abbassando ingiustamente i punteggi di `Source_Grounding` e `Qualitative_Score`. Tre esempi documentati dalle motivazioni del giudice: ha dichiarato inesistente l'Aprilia Tuono 457 (in realtà commercializzata nel 2025), inesistente la Maserati MCPura (presentata nel 2026), e ha indicato in 136 CV la potenza dell'Alfa Romeo Junior sostenendo che il valore corretto non fosse 145 CV — quando invece Stellantis ha aggiornato le schede dei mild-hybrid portando

proprio quel motore da 136 a 145 CV. In questi casi è il giudice ad avere conoscenza datata, non l'agente. Va detto, per onestà, che lo stesso giudice coglie anche allucinazioni reali dell'agente (è il caso FC3, dove l'Aprilia riceve specifiche assurde inventate): i punteggi bassi sono quindi un misto di errori veri dell'agente e di errori del giudice, e vanno pesati con questa consapevolezza. La scelta di Gemini 3.1 Flash Lite è stata dettata dal costo: la chiave gratuita di Google AI Studio concede 500 richieste al giorno, sufficienti per la valutazione senza spese; un giudice più affidabile (un modello più grande) avrebbe migliorato l'accuratezza dei punteggi ma a un costo economico non sostenibile per il progetto.

Le dimensioni dei dataset non sono particolarmente generose, danno infatti un quadro indicativo ma non statisticamente ampio, ampliare i dataset, arrivando a 50 e più prompt, renderebbe le medie più stabile e maggiormente studiabili. Inoltre, parte del punteggio basso nel Qualitative\_Score è dovuto anche alla temperatura pari a 0,6, infatti le bozze possono variare tra le esecuzioni a parità di prompt.

Di seguito vengono riportati gli screenshot direttamente dalla dashboard di LangSmith, riferiti ad entrambi i dataset, in cui è possibile osservare gli evaluator, latenze e token per run.

| # | failur...<br>0.80 AVG | quali...<br>0.56 AVG | sour...<br>0.60 AVG | tool_...<br>1.00 AVG | Latency<br>156.76 P50 | Time To Fir...<br>2.61 P50 | Total Tok...<br>135,880 TOTAL | Input To...<br>114,685 TOTAL | Output T...<br>21,195 TOTAL |
|---|-----------------------|----------------------|---------------------|----------------------|-----------------------|----------------------------|-------------------------------|------------------------------|-----------------------------|
| 1 | 1.00                  | 0.80                 | 0.70                | 1.00                 | 172.92s               |                            | 35,848                        | 30,116                       | 5,732                       |
| 2 | 1.00                  | 0.40                 | 0.60                | 1.00                 | 171.97s               |                            | 36,953                        | 30,980                       | 5,973                       |
| 3 | 1.00                  | 0.80                 | 0.80                | 1.00                 | 156.76s               |                            | 51,416                        | 43,950                       | 7,466                       |
| 4 | 0.50                  | 0.40                 | 0.20                | 1.00                 | 27.00s                |                            | 5,853                         | 4,873                        | 980                         |
| 5 | 0.50                  | 0.40                 | 0.70                | 1.00                 | 45.05s                |                            | 5,810                         | 4,766                        | 1,044                       |

| #  | failur...<br>0.86 AVG | quali...<br>0.57 AVG | sour...<br>0.71 AVG | tool_...<br>1.00 AVG | Latency<br>138.49 P50 | Time To Fir...<br>2.64 P50 | Total Tok...<br>1,024,927 TOTAL | Input To...<br>914,690 TOTAL | Output T...<br>110,237 TOTAL |
|----|-----------------------|----------------------|---------------------|----------------------|-----------------------|----------------------------|---------------------------------|------------------------------|------------------------------|
| 1  | 1.00                  | 0.80                 | 0.70                | 1.00                 | 81.76s                |                            | 29,534                          | 25,991                       | 3,543                        |
| 2  | 0.50                  | 0.40                 | 0.70                | 1.00                 | 30.67s                |                            | 6,775                           | 5,624                        | 1,151                        |
| 3  | 1.00                  | 0.80                 | 0.70                | 1.00                 | 187.03s               |                            | 59,292                          | 53,070                       | 6,222                        |
| 4  | 1.00                  | 0.60                 | 0.50                | 1.00                 | 173.63s               |                            | 45,805                          | 39,780                       | 6,025                        |
| 5  | 0.75                  | 0.80                 | 0.70                | 1.00                 | 200.46s               |                            | 50,647                          | 43,503                       | 7,144                        |
| 6  | 0.50                  | 0.40                 | 0.70                | 1.00                 | 29.81s                |                            | 6,707                           | 5,549                        | 1,158                        |
| 7  | 1.00                  | 0.60                 | 0.70                | 1.00                 | 120.82s               |                            | 53,501                          | 48,465                       | 5,036                        |
| 8  | 1.00                  | 0.60                 | 0.90                | 1.00                 | 117.98s               |                            | 46,865                          | 42,108                       | 4,757                        |
| 9  | 0.50                  | 0.40                 | 0.70                | 1.00                 | 32.18s                |                            | 6,923                           | 5,575                        | 1,348                        |
| 10 | 1.00                  | 0.80                 | 1.00                | 1.00                 | 139.46s               |                            | 49,862                          | 43,350                       | 6,512                        |
| 11 | 1.00                  | 1.00                 | 1.00                | 1.00                 | 150.24s               |                            | 63,021                          | 56,556                       | 6,465                        |
| 12 | 1.00                  | 0.40                 | 0.50                | 1.00                 | 138.49s               |                            | 41,529                          | 37,868                       | 3,661                        |
| 13 | 1.00                  | 0.60                 | 0.70                | 1.00                 | 326.80s               |                            | 68,664                          | 61,862                       | 6,802                        |
| 14 | 1.00                  | 0.60                 | 0.80                | 1.00                 | 185.69s               |                            | 49,935                          | 44,892                       | 5,043                        |
| 15 | 1.00                  | 0.40                 | 0.70                | 1.00                 | 128.06s               |                            | 33,903                          | 30,077                       | 3,826                        |
| 16 | 1.00                  | 0.40                 | 0.50                | 1.00                 | 232.01s               |                            | 72,610                          | 66,751                       | 5,859                        |
| 17 | 0.75                  | 0.80                 | 0.90                | 1.00                 | 165.21s               |                            | 57,651                          | 51,022                       | 6,629                        |
| 18 | 1.00                  | 0.40                 | 0.60                | 1.00                 | 128.10s               |                            | 28,936                          | 25,229                       | 3,707                        |
| 19 | 0.50                  | 0.40                 | 0.70                | 1.00                 | 32.13s                |                            | 6,819                           | 5,599                        | 1,220                        |
| 20 | 0.50                  | 0.40                 | 0.70                | 1.00                 | 27.16s                |                            | 6,213                           | 5,299                        | 914                          |
| 21 | 0.50                  | 0.40                 | 0.70                | 1.00                 | 29.23s                |                            | 6,799                           | 5,666                        | 1,133                        |
| 22 | 1.00                  | 0.60                 | 0.60                | 1.00                 | 314.83s               |                            | 67,174                          | 60,658                       | 6,516                        |
| 23 | 1.00                  | 0.20                 | 0.50                | 1.00                 | 227.90s               |                            | 50,663                          | 44,044                       | 6,619                        |
| 24 | 1.00                  | 0.80                 | 0.90                | 1.00                 | 192.59s               |                            | 64,571                          | 60,980                       | 3,591                        |
| 25 | 1.00                  | 0.60                 | 0.60                | 1.00                 | 125.27s               |                            | 50,528                          | 45,172                       | 5,356                        |

## Conclusioni

Il progetto ha realizzato un agente editoriale automotive completo, che copre l'intero flusso dalla richiesta dell'utente alla pubblicazione di un articolo corredato di fonti, copertina e registrazione nel Knowledge Graph. Tutti i requisiti della traccia sono stati soddisfatti e in più punti superati: il sistema implementa un processo di ragionamento agentico ReAct con passi espliciti (Thought → Action → Observation), scelta dinamica degli strumenti e giustificazione dell'uso dei tool nella traccia; integra gli strumenti richiesti, ricerca web (incapsulata in un servizio MCP dedicato), RAG su documenti locali e Knowledge Graph in lettura e scrittura, più diversi strumenti aggiuntivi, tra cui il confronto tra veicoli basato sul modello fine-tuned, oltre a recupero schede tecniche, trend RSS, analisi SEO e generazione delle copertine. A questi si aggiungono la pianificazione editoriale, una gestione esplicita dello stato che attraversa tutto il grafo, e un Human-in-the-loop articolato su cinque punti di controllo.

Il fine-tuning del modello comparatore (QLoRA su Llama 3.2 1B) ha prodotto un miglioramento netto e coerente su tutte le metriche misurate: semantica, lessico, caratteri, fluenza, aderenza al formato e qualità dei giudizi, verificato in modo rigoroso con un confronto su un test set di veicoli mai visti. La valutazione dell'agente sui due dataset ha mostrato un

uso degli strumenti sempre corretto, zero errori di esecuzione e un grounding crescente con l'ampliarsi del dataset, confermando la robustezza dell'architettura.

Il filo conduttore dell'intero lavoro è stato imparare a costruire un sistema affidabile attorno a un modello locale piccolo. Da qui derivano le lezioni più importanti. La prima è che, dove la scelta corretta è certa, conviene renderla deterministica anziché affidarla al modello: i guardrail e le regole tassative sono il modo corretto di usare un 3B, lasciandogli il giudizio dove serve davvero e togliendogli le decisioni che sbaglierebbe. La seconda è che il prompt va calibrato: più volte un prompt troppo prescrittivo ha peggiorato il comportamento, perché il modello piccolo tende a imitare un formato invece di eseguirlo. La terza è il valore dell'osservabilità: quasi ogni problema è stato diagnosticato grazie alle tracce di LangSmith, che mostravano esattamente cosa il modello faceva passo per passo. Infine, sul piano dell'esecuzione locale, il compromesso tra dimensione del modello e finestra di contesto si è rivelato decisivo: un modello più piccolo ma con contesto ampio si è dimostrato più affidabile, a parità di hardware, di uno più grande ma con l'input troncato.

## Limiti e sviluppi futuri

Il sistema mantiene limiti riconosciuti con onestà. La scelta degli strumenti da parte del modello, pur molto migliorata, resta probabilistica e non perfetta. La valutazione è condizionata da due fattori: gli evaluator sono tarati sul post completo e penalizzano le richieste di sole proposte o quelle che si fermano ai gate umani, e il giudice LLM (Gemini 3.1 Flash Lite) ha mostrato a sua volta allucinazioni su modelli molto recenti, rendendo alcuni punteggi pessimistici. Il modello comparatore fine-tuned, infine, mostra un bias verso il pareggio nei confronti più sfumati: ha imparato ottimamente formato e stile, ma la correttezza del giudizio nei casi vicini è ancora migliorabile. Sul piano pratico, l'esecuzione interamente locale comporta latenze nell'ordine dei minuti per articolo, e la rete di sicurezza del RAG può iniettare occasionalmente un frammento marginale su temi non coperti dai documenti locali.

Sul fine-tuning, ampliare il dataset, ed in particolare con esempi che premiano il decretare un vincitore netto e bilancino la frequenza dei pareggi, affronterebbe direttamente il bias del giudice. Sulla valutazione, adottare un giudice più affidabile (un modello più capace, accettandone il costo). Sul Knowledge Graph, assegnare ai post di confronto un topic più specifico (i veicoli confrontati) invece della categoria generica eviterebbe la contaminazione tra confronti diversi, e salvare le fonti come URL puliti anziché come output grezzi dei tool ridurrebbe il rumore in scrittura. Infine, su un'infrastruttura con GPU dedicate o modelli serviti via API, le latenze scenderebbero drasticamente, rendendo il sistema utilizzabile in un contesto editoriale reale.

Nel complesso, il progetto dimostra che è possibile costruire un agente editoriale autonomo, ancorato alle fonti e supervisionato dall'uomo interamente con modelli locali e strumenti a basso costo, a patto di progettare con attenzione il confine tra ciò che si affida al modello e ciò che si governa in modo deterministico.



